SMART FUEL DISPATCH & FLEET TELEMETRY SYSTEM

23CS54C – MODERN WEB TECHNOLOGIES

Submitted by

PRAVIN N (24104080)

In partial fulfillment for the award of the degree
of
BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY



NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)
K.R.NAGAR, KOVILPATTI - 628503

OCTOBER - 2026



NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)
K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "Smart Fuel Dispatch & Fleet Telemetry System", is the bonafide work of PRAVIN N (24104080) who carried out the project work under my supervision.

Course Instructor / Guide

Submitted to the 23CS54C – MODERN WEB TECHNOLOGIES Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner **Co-Examiner / External Examiner**

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	**INTRODUCTION**	**5**
2	OBJECTIVES	7
3	DESCRIPTION	9
4	CLASS DIAGRAM	12
5	TABLE STRUCTURES	14
6	ER DIAGRAM	23
7	TECH STACK	28
8	**MODULES**	**32**
9	**CONCLUSION**	**58**
-	REFERENCES	60

CHAPTER 1
INTRODUCTION

Fuel distribution and bulk petroleum dispatch represent time-critical supply chain operations where order processing, resource allocation, inventory levels, route telemetry, and delivery visibility must operate in complete synchronization. Traditional fuel distribution setups often depend on disconnected manual procedures, including phone calls, paper delivery slips, and isolated spreadsheet logs. When these activities are coordinated without an integrated digital backbone, it becomes exceptionally difficult for dispatchers and station managers to know which tanker is carrying an order, where the driver currently is along the transit corridor, whether sufficient stock is available in the depot storage tanks, and whether an order has reached the customer site securely.

The Smart Fuel Dispatch & Fleet Telemetry System is a modern full-stack web application engineered to solve these operational challenges for bulk fuel delivery logistics. Known internally within the codebase and user interface as the Fuel Delivery Management System (FDMS), the platform consolidates corporate customer onboarding, KYC document verification, commercial fuel order placement, depot manager approvals, automated inventory reservation, tanker vehicle and driver assignment, smart dispatch routing, live GPS telemetry tracking, underground tank telemetry, multi-product pricing engines, real-time alerts, immutable audit logging, multi-role analytics dashboards, and an operational AI logistics assistant into a single unified web platform.

The frontend architecture is implemented with React 18 and Vite, delivering an ultra-responsive, component-driven single-page application (SPA). React Router DOM manages client-side navigation and role-aware protected route guards. Global application state is orchestrated using React's Context API with useReducer patterns, while Axios handles authenticated REST communication via automatic Bearer token interceptors. Real-time telemetry and instant operational notifications are powered by Socket.IO client connections. Interactive fleet positioning and route corridors are rendered using Leaflet and React-Leaflet with OpenStreetMap cartographic tiles. The user interface features a modern design system with dark and light themes and complete internationalization supporting English, Tamil, and Hindi.

The backend server is implemented using Node.js and Express 5, architected around modular controllers and domain-specific route gateways. Persistent data storage is managed using MongoDB Atlas and the Mongoose Object Data Modeling (ODM) library, utilizing schema validation, compound indexes, virtual getters, and pre-save hooks. Authentication security is enforced via JSON Web Tokens (JWT) and bcryptjs cryptographic password hashing, complemented by Google OAuth 2.0 verification and SMS-ready one-time password (OTP) verification. Multipart KYC document uploads are processed securely via Multer middleware. A Socket.IO server gateway broadcasts real-time telemetry coordinates and targeted event notifications to role-based rooms.

The platform natively accommodates six distinct user roles: Administrator, Depot Manager, Customer, Driver, Support Executive, and Auditor. In addition, the codebase includes an automated continuous integration pipeline configured via Jenkins (`Jenkinsfile`) and production-grade cloud deployment configurations via Vercel (`vercel.json`), backed by nine automated Node.js validation test suites. This report provides an exhaustive, academically rigorous record of the architectural design, schema models, module implementations, telemetry workflows, and experimental results.

CHAPTER 2
OBJECTIVES

The primary purpose of the Smart Fuel Dispatch & Fleet Telemetry System is to design, develop, and evaluate a reliable, modern web platform that replaces fragmented manual coordination with an automated, role-aware, and real-time logistics solution. The project accomplishes the following eleven key technical objectives:

Digitize Bulk Fuel Delivery Operations: Replace fragmented paper delivery slips, manual phone dispatching, and manual spreadsheets with a centralized, automated web platform that unifies all stakeholder interactions.

Role-Based Access Control & Multi-Modal Authentication: Implement secure authentication supporting email/password with bcrypt hashing, Google OAuth 2.0 single sign-on, and mobile OTP login, enforced across six distinct user roles using JWT and route protection middleware.

Verified Corporate Onboarding & KYC Workflow: Provide a multi-step registration workflow that captures company information, GSTIN, PAN, and fuel requirements with multipart document uploads, requiring administrative review before granting access.

End-to-End Fuel Order Lifecycle Management: Manage the full lifecycle of commercial fuel orders—including creation, dynamic pricing, depot approval, stock reservation, tanker assignment, transit tracking, delivery proof submission, and automated PDF invoicing.

Intelligent Dispatch & Proximity Recommendations: Implement a smart dispatch recommendation engine utilizing the Haversine great-circle formula to calculate Euclidean road proximity, recommending the nearest depot and tanker to optimize delivery timelines.

Real-Time Fleet Telemetry & GPS Tracking: Capture periodic GPS coordinate pings (latitude, longitude, speed, heading, timestamp) via REST and Socket.IO, streaming live vehicle positions to an interactive Leaflet map and preserving historical breadcrumb trails.

Depot Tank Inventory Telemetry & Low-Stock Alerts: Monitor depot fuel storage vaults with real-time tracking of capacity, current volume, reserved stock, temperature, and pressure, automatically triggering threshold alerts and locking orders when stock is depleted.

Dynamic Multi-Product Pricing & Taxation Engine: Provide centralized rate configuration for Diesel, Petrol, Premium Diesel, and CNG, calculating base fuel charges, 18% GST, and delivery freight dynamically across all orders and formal quotation requests.

Real-Time Notification Gateway & Immutable Audit Trail: Broadcast instant event alerts across role-specific Socket.IO channels and maintain an append-only activity log that captures user ID, role, action, target entity, timestamp, and IP address for compliance.

Executive Dashboards, Internationalization & AI Assistant: Provide tailored analytical dashboards for all roles, complete multilingual support for English, Tamil, and Hindi, a responsive dark/light visual theme, and a context-aware logistics query assistant.

Automated CI/CD Deployment Pipeline & Verification Suite: Establish an automated build and test pipeline via Jenkins, serverless multi-service deployment via Vercel, and nine dedicated Node.js test scripts validating authorization, state lifecycles, and telemetry.

CHAPTER 3
DESCRIPTION

The Smart Fuel Dispatch & Fleet Telemetry System is architected as a layered full-stack web application structured around the Model-View-Controller (MVC) paradigm. The React client delivers an interactive presentation layer, the Node.js/Express server provides RESTful microservices and a WebSocket gateway, and MongoDB Atlas serves as the persistent document store. The operational surface of the platform is partitioned into eight comprehensive functional areas:

1. Authentication and Access Control

The authentication subsystem guarantees identity verification and access segregation across the platform. Users authenticate using standard credentials (email and password), Google OAuth 2.0 social sign-in, or mobile OTP. Passwords are cryptographically salted and hashed using bcryptjs. Upon successful authentication, the server signs a JSON Web Token (JWT) encapsulating the user ID, email, and assigned role. On the client side, AuthContext maintains session state, while ProtectedRoute wrappers redirect unauthenticated visitors and restrict portal views based on role permissions. On the server side, `protect` and `authorize` middleware validate incoming Bearer tokens and enforce role permissions across sensitive endpoints. Rejected or pending corporate accounts are strictly denied login.

2. Customer and Registration Management

Corporate fuel buyers must undergo a formal vetting process before purchasing fuel. The registration workflow collects corporate identity, trade license numbers, GSTIN, PAN, registered office address, contact coordinates, and projected fuel consumption. Applicants upload verification documents (business registration certificates, GST filings, PAN cards) via multipart form uploads handled by Multer. Submissions enter a 'Pending' queue in the database. Authorized Depot Managers or Administrators review the application through the Applications Review Portal. Approving an application automatically provisions an active User account, establishes the Customer master record, and dispatches an acceptance notification.

3. Order and Dispatch Management

Commercial fuel orders represent the core transaction within the system. An order document records the customer ID, selected fuel type (Diesel, Petrol, Premium Diesel, CNG), requested volume in liters, delivery site coordinates, scheduled delivery window, assigned driver, assigned tanker vehicle, itemized financials, and lifecycle timestamps. When an order is submitted, the system calculates pricing against active FuelPricing tariffs, including an 18% GST levy and delivery freight. Upon Depot Manager approval, the requested fuel volume is atomically reserved in the corresponding depot tank. When the order is dispatched, the assigned tanker and driver transition to active duty. Upon delivery verification, stock is formally deducted from the vault, and a digital invoice is finalized.

4. Driver and Fleet Management

The fleet management module maintains operational oversight of transportation assets and personnel. Driver entities record full name, phone number, assigned vehicle number, duty status (Available, On Duty, Inactive), total completed trips, commercial driver license number, and email. Tanker vehicle records track registration number, assigned driver, operational status (Available, In Transit, Maintenance, Out of Service), fuel capacity, fuel cargo type, current fuel level, tyre pressure, engine temperature, destination coordinates, estimated time of arrival (ETA), and last known latitude and longitude. Fleet managers can register new tankers, inspect diagnostic telemetry, and adjust duty rosters in real time.

5. GPS and Telemetry Management

Location telemetry flows continuously into the platform via REST API ingestion and WebSocket events. Telemetry packets containing vehicle ID, driver ID, latitude, longitude, speed (km/h), heading (degrees), and timestamp are validated by the backend. Each ping updates the live coordinates on the primary Vehicle document and is simultaneously written as an immutable record in the VehicleLocation and DriverLocation historical collections. The Socket.IO server broadcasts the coordinates to connected client rooms, causing the interactive Leaflet Fleet Map to update smoothly without page reloading. Historical trails store up to 1,000 points per vehicle for route analysis.

6. Inventory and Fuel Pricing

Depot storage vaults are represented as tank documents storing fuel code, name, total capacity (liters), current physical stock, reserved stock (allocated to approved pending orders), minimum safety threshold, temperature (°C), pressure (PSI), and refill logs. Virtual Mongoose getters automatically compute available free stock (`currentStock - reservedStock`), fill percentage, and low-stock warning status. The pricing subsystem manages base prices per liter, standard tax percentages (18% GST), and delivery charges across all fuel categories, ensuring quotes and order bills reflect accurate commercial rates.

7. Notifications, Audit Logs and Reports

Operational awareness and regulatory governance are maintained through real-time notifications and immutable audit logging. System events—such as new registrations, order approvals, dispatch departures, delivery completions, and tank threshold warnings—create notification documents pushed immediately to role-specific Socket.IO rooms. The audit logging engine captures every critical state change, logging the performing user ID, user role, action type (e.g., Login, Order Approved, Tank Refilled), affected entity ID, IP address, and timestamp. The reporting module aggregates transactional data to produce revenue summaries, consumption charts, and CSV/Excel exports.

8. Dashboard, Settings, Localisation and Assistant

Role-tailored dashboards provide executive, managerial, and operational overviews customized to each stakeholder role. The Settings portal enables individual users to configure visual themes (dark or light mode), notification sound and alert preferences, and interface language. Localisation is implemented using client-side translation dictionaries supporting

English, Tamil, and Hindi. Finally, an interactive floating Fuel Logistics Assistant analyzes active order states, inventory levels, and fleet availability to provide rapid answers to common operational inquiries and offer role-specific quick action shortcuts.

CHAPTER 4
CLASS DIAGRAM

The domain structure of the Smart Fuel Dispatch & Fleet Telemetry System is structured using object-oriented principles mapped onto MongoDB document collections via Mongoose models. Figure 4.1 illustrates the comprehensive Class Diagram representing the core domain models, their internal attributes, methods, and relationships.

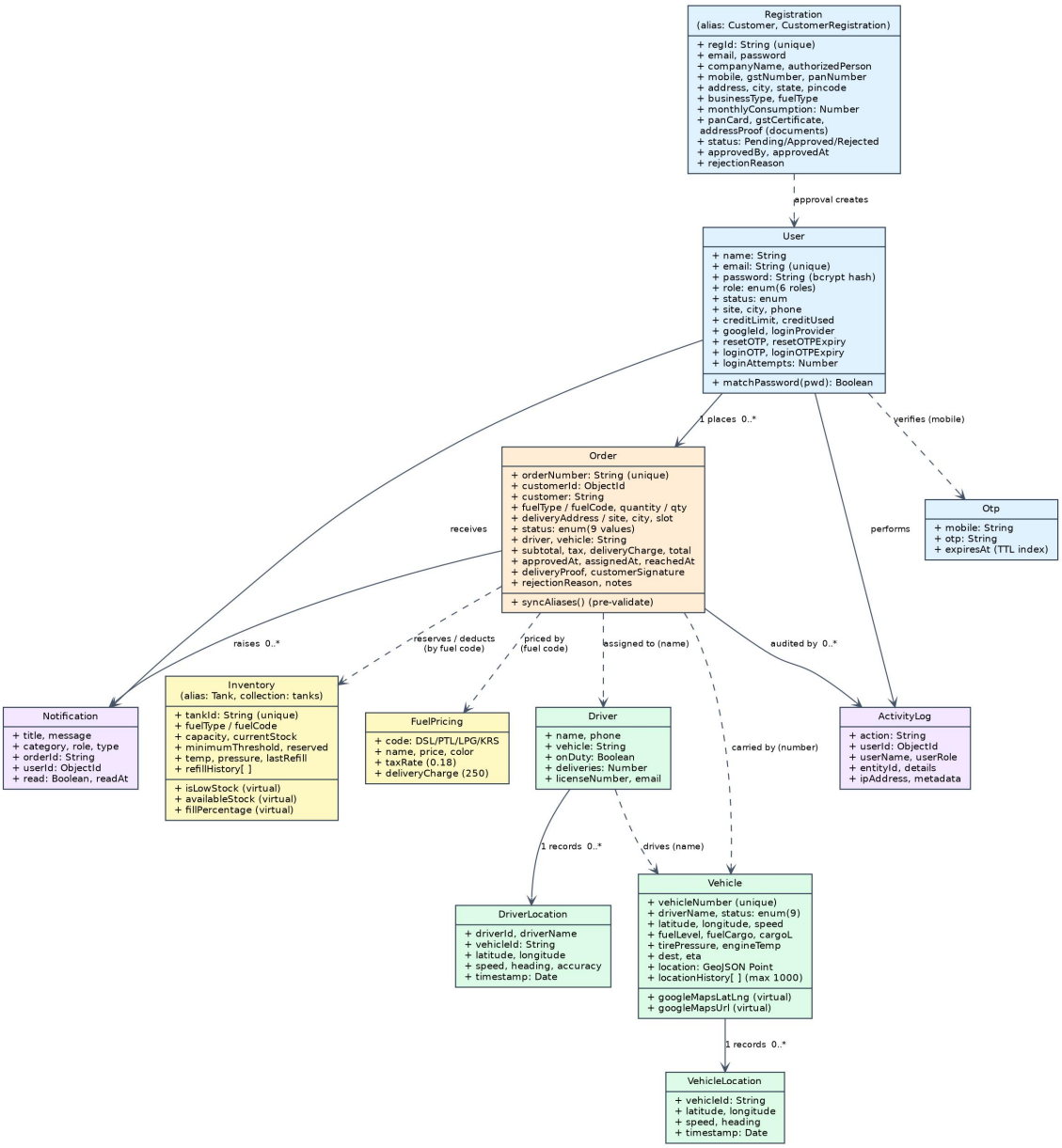


Figure 4.1: Class Diagram of the Smart Fuel Dispatch & Fleet Telemetry System

As depicted in Figure 4.1, the class structure cleanly decouples master entities, operational transactions, and time-series telemetry. The `Order` class forms the central transactional entity, connecting the `User` (acting as customer), the assigned `Driver`, and the dispatched `Vehicle`. Solid association arrows represent referential integrity enforced via MongoDB

ObjectIds (e.g., `Order.customerId` referencing `User._id`, and `Notification.userId` referencing `User._id`). Dashed arrows indicate logical associations resolved programmatically within controller business logic using unique identifiers, such as linking an order to a `Driver` by name and to a `Vehicle` by registration plate number.

The `User` model represents all authenticated actors across the six system roles (Admin, Depot Manager, Customer, Driver, Support Executive, Auditor). Corporate customer applicants are represented by `Registration` (aliased as `Customer` and `CustomerRegistration`), which encapsulates company identity, GSTIN/PAN data, and uploaded document paths. The `Vehicle` and `Driver` classes represent physical delivery resources, while `VehicleLocation` and `DriverLocation` isolate historical telemetry pings from the active master records to maintain query responsiveness. `Inventory` (aliased as `Tank`) monitors underground fuel vaults, and `FuelPricing` manages commercial rates. Finally, `Notification`, `ActivityLog`, and `Otp` provide communication, auditability, and multi-factor authentication services across the system.

CHAPTER 5
TABLE STRUCTURES

The Smart Fuel Dispatch & Fleet Telemetry System utilizes MongoDB Atlas as its primary database management system. In MongoDB, data models are defined using Mongoose schemas and stored as collections of flexible BSON documents. Every document automatically receives an immutable primary key `_id` of type ObjectId, as well as automatic `createdAt` and `updatedAt` audit timestamps where `timestamps: true` is configured in the schema options. The tables below specify the field definitions, data types, required validations, and constraints for all twelve schemas implemented in the project.

1. User Schema (Collection: users)

Stores credentials, personal profiles, contact details, assigned roles, and approval statuses for all system users.

Field	Type	Required	Default / Constraints
name	String	Yes	Trimmed string; display name
email	String	Yes	Unique, lowercase, trimmed; primary login identifier
password	String	No	Bcrypt hash; optional for OAuth/OTP accounts
phone	String	No	Trimmed; mobile number for OTP/SMS
company	String	No	Trimmed; associated corporate organization
role	String	Yes	Enum: Admin, Depot Manager, Customer, Driver, Support Executive, Auditor (Default: Customer)
status	String	Yes	Enum: Active, Inactive, Pending Approval, Rejected (Default: Active)
googleId	String	No	Google OAuth 2.0 unique subject identifier
isApproved	Boolean	Yes	Default true for staff; false for pending customers
avatar	String	No	URL string to profile image
lastLogin	Date	No	Timestamp of most recent authentication session
createdAt	Date	Auto	Managed timestamp; index for recency queries

Field	Type	Required	Default / Constraints
updatedAt	Date	Auto	Managed timestamp

2. Order Schema (Collection: orders)

Encapsulates bulk fuel delivery orders, customer references, quantities, financials, assigned assets, and lifecycle status.

Field	Type	Required	Default / Constraints
orderNumber	String	No	Unique generated identifier; format ORD-YYMMDD-NNN
customerId	ObjectId	Yes	Reference to users collection; indexed
customerName	String	Yes	Denormalized customer company name for query speed
customerPhone	String	No	Customer contact number
fuelType	String	Yes	Enum: Diesel, Petrol, Premium Diesel, CNG; alias fuelCode
quantity	Number	Yes	Ordered volume in liters; minimum 100 L; alias qty
deliveryAddress	String	Yes	Destination site address; alias site
city	String	Yes	Target delivery city/region in Tamil Nadu
deliverySlot	String	No	Enum: Morning (06-12), Afternoon (12-18), Evening (18-22)
status	String	Yes	Enum: Pending, Approved, Assigned, Dispatched, Delivered, Cancelled, Rejected (Default: Pending)
assignedDriver	String	No	Denormalized driver name; default null
assignedTanker	String	No	Vehicle registration number (e.g., TN-72-AB-1234); default null
basePrice	Number	Yes	Unit tariff per liter fetched from FuelPricing
taxAmount	Number	Yes	18% GST statutory amount
deliveryCharge	Number	Yes	Calculated freight delivery charge

Field	Type	Required	Default / Constraints
totalAmount	Number	Yes	Total invoice amount in INR; index for reporting
paymentStatus	String	Yes	Enum: Pending, Paid, COD, Invoiced (Default: Pending)

Note: A Mongoose pre-validate hook ensures that alias field pairs (`fuelType`/`fuelCode`, `quantity`/`qty`, `deliveryAddress`/`site`) remain perfectly synchronized across client payloads.

3. Driver Schema (Collection: drivers)

Manages fleet drivers, license documentation, current availability, and assigned vehicles.

Field	Type	Required	Default / Constraints
name	String	Yes	Driver full name; trimmed
phone	String	Yes	Mobile contact number; unique indexed
email	String	No	Contact email address
assignedVehicle	String	No	Tanker registration plate number; default null
status	String	Yes	Enum: Available, On Duty, Inactive, On Leave (Default: Available)
licenseNumber	String	Yes	Heavy vehicle commercial license identifier
totalTrips	Number	No	Completed delivery trip counter (Default: 0)
currentLocation	Object	No	Subdocument: { lat: Number, lng: Number, timestamp: Date }

4. Vehicle Schema (Collection: vehicles)

Maintains tanker vehicle master records, diagnostic sensor parameters, and real-time transit state.

Field	Type	Required	Default / Constraints
vehicleNumber	String	Yes	Unique registration plate (e.g., TN-69-X-9988); alias vehicle
driverName	String	No	Currently assigned driver name

Field	Type	Required	Default / Constraints
status	String	Yes	Enum: Available, In Transit, Maintenance, Out of Service (Default: Available)
cargoType	String	Yes	Enum: Diesel, Petrol, Premium Diesel, CNG, Multi-Chamber
capacity	Number	Yes	Tanker gross capacity in liters (e.g., 10,000 to 24,000 L)
fuelLevel	Number	Yes	Current liquid level percentage (0 to 100%)
tyrePressure	Number	No	Diagnostic tyre pressure in PSI (nominal: 110 PSI)
engineTemp	Number	No	Diagnostic engine coolant temperature in °C
latitude	Number	Yes	Current GPS latitude coordinate
longitude	Number	Yes	Current GPS longitude coordinate
speed	Number	No	Current road speed in km/h
destination	String	No	Assigned drop-off location or customer site
estimatedTime	String	No	Computed ETA string (e.g., '45 mins')

Note: The schema implements two virtual getters, `googleMapsLatLng` and `googleMapsUrl`, which expose coordinates for mapping integrations.

5. VehicleLocation Schema (Collection: VehicleLocation)

Stores historical GPS breadcrumb pings recorded from tanker vehicles for trail rendering and speed auditing.

Field	Type	Required	Default / Constraints
vehicleId	String	Yes	Tanker registration number; indexed
latitude	Number	Yes	Recorded latitude coordinate
longitude	Number	Yes	Recorded longitude coordinate
speed	Number	No	Instantaneous road velocity in km/h
heading	Number	No	Compass heading angle in degrees (0 - 359°)

Field	Type	Required	Default / Constraints
timestamp	Date	Yes	Hardware telemetry timestamp; compound index with vehicleId

6. DriverLocation Schema (Collection: DriverLocations)

Tracks driver-centric mobile GPS location logs when drivers transmit pings independently of onboard vehicle modems.

Field	Type	Required	Default / Constraints
driverId	Mixed	No	Driver identifier (ObjectId or numeric string); indexed
vehicleId	String	No	Associated vehicle plate number; indexed
latitude	Number	Yes	Driver device latitude coordinate
longitude	Number	Yes	Driver device longitude coordinate
timestamp	Date	Yes	Telemetry capture timestamp (Default: Date.now)

7. Inventory Schema (Collection: tanks)

Maintains bulk fuel storage vaults, physical stock volumes, reserved quantities, and environmental telemetry.

Field	Type	Required	Default / Constraints
fuelType	String	Yes	Fuel category: Diesel, Petrol, Premium Diesel, CNG; alias fuelCode
tankName	String	Yes	Vault name/number (e.g., Underground Vault Alpha-1)
capacity	Number	Yes	Vault physical volume capacity in liters (e.g., 50,000 L)
currentStock	Number	Yes	Current gross liquid inventory in liters
reservedStock	Number	Yes	Volume committed to approved pending dispatches (Default: 0)
minThreshold	Number	Yes	Safety threshold triggering low-stock alert (e.g., 10,000 L)

Field	Type	Required	Default / Constraints
temperature	Number	No	Ambient vault temperature in °C (Default: 28°C)
pressure	Number	No	Underground chamber vapor pressure in PSI (Default: 14.7 PSI)
lastRefillDate	Date	No	Timestamp of most recent bulk tanker decanting
lastRefillAmount	Number	No	Volume added during last refill in liters

Note: Virtual fields `availableStock` (`currentStock - reservedStock`), `fillPercentage`, and `isLowStock` are calculated dynamically on document read.

8. FuelPricing Schema (Collection: fuelpricings)

Maintains tariff rates, tax levies, and delivery freight rates for all fuel types.

Field	Type	Required	Default / Constraints
code	String	Yes	Fuel code: DSL, PTL, PDSL, CNG; unique indexed
name	String	Yes	Descriptive name: Diesel, Petrol, Premium Diesel, CNG
pricePerLiter	Number	Yes	Commercial base rate per liter in INR (e.g., 89.50)
taxRate	Number	Yes	Statutory tax percentage (Default: 18 for 18% GST)
deliveryFee	Number	Yes	Standard freight charge per trip in INR (Default: 500)
lastUpdated	Date	Auto	Timestamp of last tariff adjustment (Default: Date.now)

9. Notification Schema (Collection: notifications)

Stores targeted event notifications pushed to individual users or broadcast across role rooms.

Field	Type	Required	Default / Constraints
userId	ObjectId	No	Target user identifier; null for role-wide broadcasts; indexed
role	String	No	Target audience role: Admin, Depot Manager, Customer, Driver

Field	Type	Required	Default / Constraints
title	String	Yes	Notification headline (e.g., 'Order Approved')
message	String	Yes	Descriptive alert message with order/stock context
type	String	Yes	Enum: info, success, warning, alert (Default: info)
read	Boolean	Yes	Read status flag (Default: false); indexed
createdAt	Date	Auto	Creation timestamp; indexed descending for recent feeds

10. ActivityLog Schema (Collection: activitylogs)

Maintains an immutable audit trail of system events, security operations, and transactional approvals.

Field	Type	Required	Default / Constraints
userId	ObjectId	No	Identifier of user executing the action; indexed
userName	String	No	Display name of performing user
userRole	String	No	Role of performing user at time of action execution
action	String	Yes	Action description: Login, Order Created, Status Change, Tank Refill
entityType	String	No	Target domain collection: Order, User, Inventory, Vehicle
entityId	String	No	String representation of target entity identifier
ipAddress	String	No	Client network IP address
createdAt	Date	Auto	Timestamp; indexed descending; compound indexed with action

11. Otp Schema (Collection: otps)

Stores short-lived one-time verification passcodes for multi-factor SMS authentication.

Field	Type	Required	Default / Constraints
mobile	String	Yes	Target phone number; indexed

Field	Type	Required	Default / Constraints
otp	String	Yes	6-digit cryptographic verification passcode
createdAt	Date	Auto	Creation timestamp; configured with MongoDB TTL expiry index

Note: The schema utilizes MongoDB's Time-To-Live (TTL) index feature on `createdAt` with `expires: 300`, causing passcodes to automatically self-purge after 5 minutes.

12. Registration Schema (Collection: customerregistrations)

Captures corporate customer onboarding applications, statutory business identifiers, and KYC document uploads.

Field	Type	Required	Default / Constraints
regId	String	No	Unique application code; format REG-YYYY-NNNN
companyName	String	Yes	Registered corporate trade name; trimmed
contactPerson	String	Yes	Authorized representative full name
email	String	Yes	Official corporate contact email; indexed
phone	String	Yes	Authorized mobile phone number
gstNumber	String	Yes	15-character statutory GSTIN tax identifier
panNumber	String	Yes	10-character corporate Permanent Account Number (PAN)
address	String	Yes	Registered physical corporate headquarters address
city	String	Yes	Operating city / dispatch region
fuelRequirement	String	Yes	Estimated monthly fuel requirement (e.g., '15,000 Liters / Month')
documents	Array	No	Array of uploaded file metadata: { fileName, filePath, uploadDate }
status	String	Yes	Enum: Pending, Approved, Rejected (Default: Pending); indexed

Field	Type	Required	Default / Constraints
reviewedBy	ObjectId	No	Reference to reviewing Depot Manager or Admin user
reviewNotes	String	No	Reviewer justification notes or rejection reasons

CHAPTER 6
ER DIAGRAM

The Entity-Relationship (ER) design of the Smart Fuel Dispatch & Fleet Telemetry System is organized into three logical data tiers: Master Entities, Transactional Entities, and Time-Series Telemetry Entities. Master entities (User, Registration, Driver, Vehicle, Inventory, FuelPricing) define static and semi-static organizational assets. Transactional entities (Order, Notification, ActivityLog, Otp) capture operational events and business workflows. Time-series entities (VehicleLocation, DriverLocation) record high-frequency spatial streams, intentionally isolated from core records to safeguard query throughput. Figure 6.1 illustrates the Core ER Diagram.

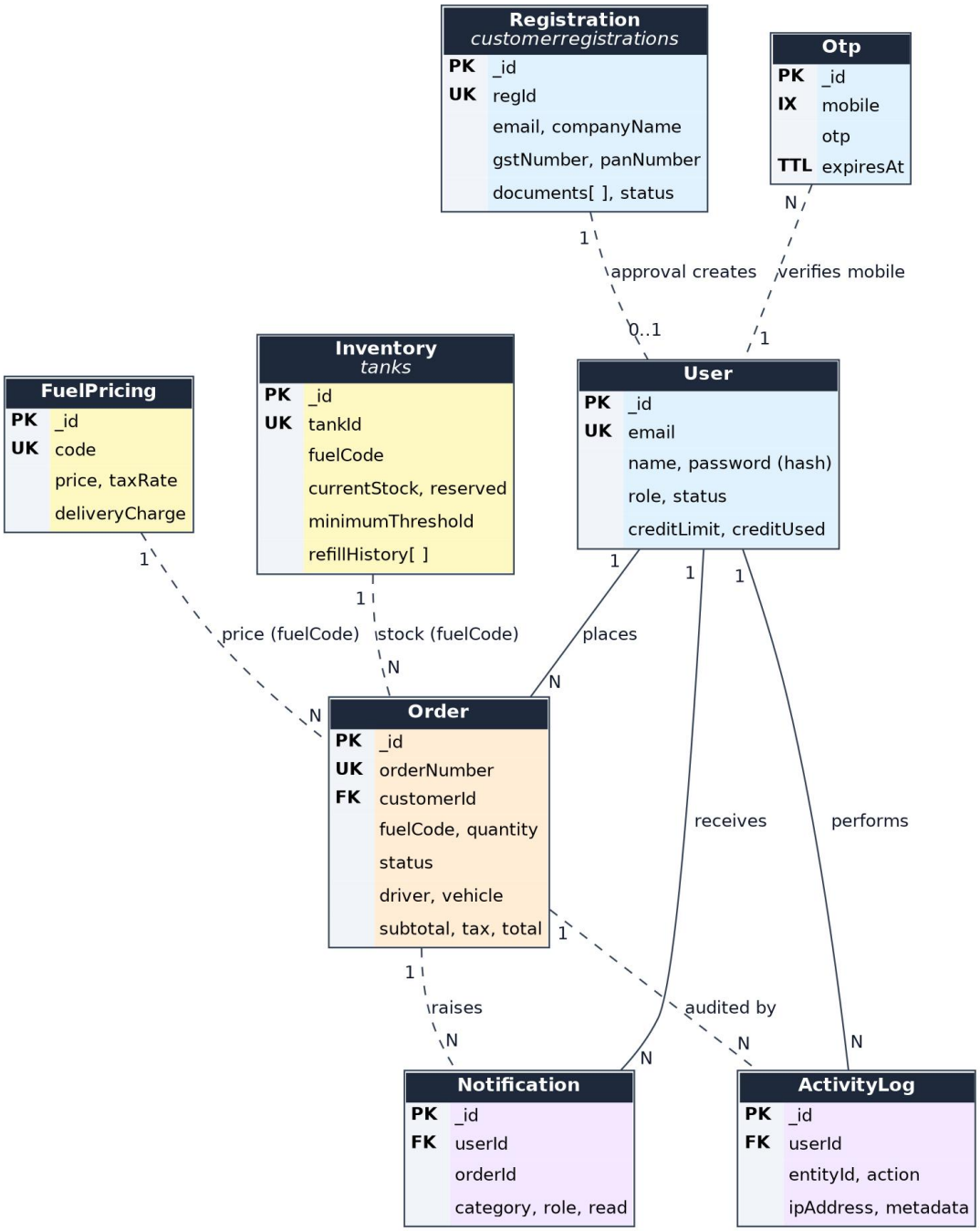


Figure 6.1: Core Entity-Relationship Diagram (PK = Primary Key, FK = Foreign Reference, IX = Index)

In MongoDB's document-oriented architecture, relationships are implemented via a combination of ObjectId references and denormalized snapshots. Table 6.1 details the complete cardinality and referencing mechanics established across the system models.

Entity A	Cardinality	Entity B	Linking Field	Kind	Description
User	1 : N	Order	customerId	Reference	A customer user places multiple commercial fuel delivery orders.

Entity A	Cardinality	Entity B	Linking Field	Kind	Description
User	1 : N	Notification	userId	Reference	Notifications targeted directly to an individual user account.
User	1 : N	ActivityLog	userId	Reference	Audit log capturing the specific user who executed an operation.
Registration	1 : 1	User	email	Logical	Approving a registration provisions an active User account with role Customer.
Order	N : 1	FuelPricing	fuelType / code	Logical	Orders reference fuel code to calculate unit price and 18% GST.
Order	N : 1	Inventory	fuelType / tank	Logical	Order approvals atomically reserve stock in the matching depot tank.
Driver	1 : 1	Vehicle	assignedVehicle	Logical	A driver is assigned to one primary tanker vehicle registration.
Order	N : 1	Driver	assignedDriver	Snapshot	Orders record driver name at dispatch; remains immutable for history.
Order	N : 1	Vehicle	assignedTanker	Snapshot	Orders record vehicle registration number at dispatch time.
Vehicle	1 : N	VehicleLocation	vehicleId	Foreign Key	A vehicle produces an append-only sequence of GPS location pings.
Driver	1 : N	DriverLocation	driverId	Foreign Key	A driver produces an append-only stream of mobile GPS pings.

Entity A	Cardinality	Entity B	Linking Field	Kind	Description
Depot Tank	1 : N	ActivityLog	entityId	Logical	Tank decanting and threshold alarms are logged for auditing.
User (Reviewer)	1 : N	Registration	reviewedBy	Reference	Depot manager or Admin reviewing KYC applications.
User (Phone)	1 : N	Otp	mobile	Logical	Mobile passcodes linked temporarily to phone during OTP sign-in.

Figure 6.2 illustrates the specialized Fleet and Telemetry Entity Relationship topology, depicting how high-frequency GPS coordinates integrate with dispatch transactions while preserving system responsiveness.

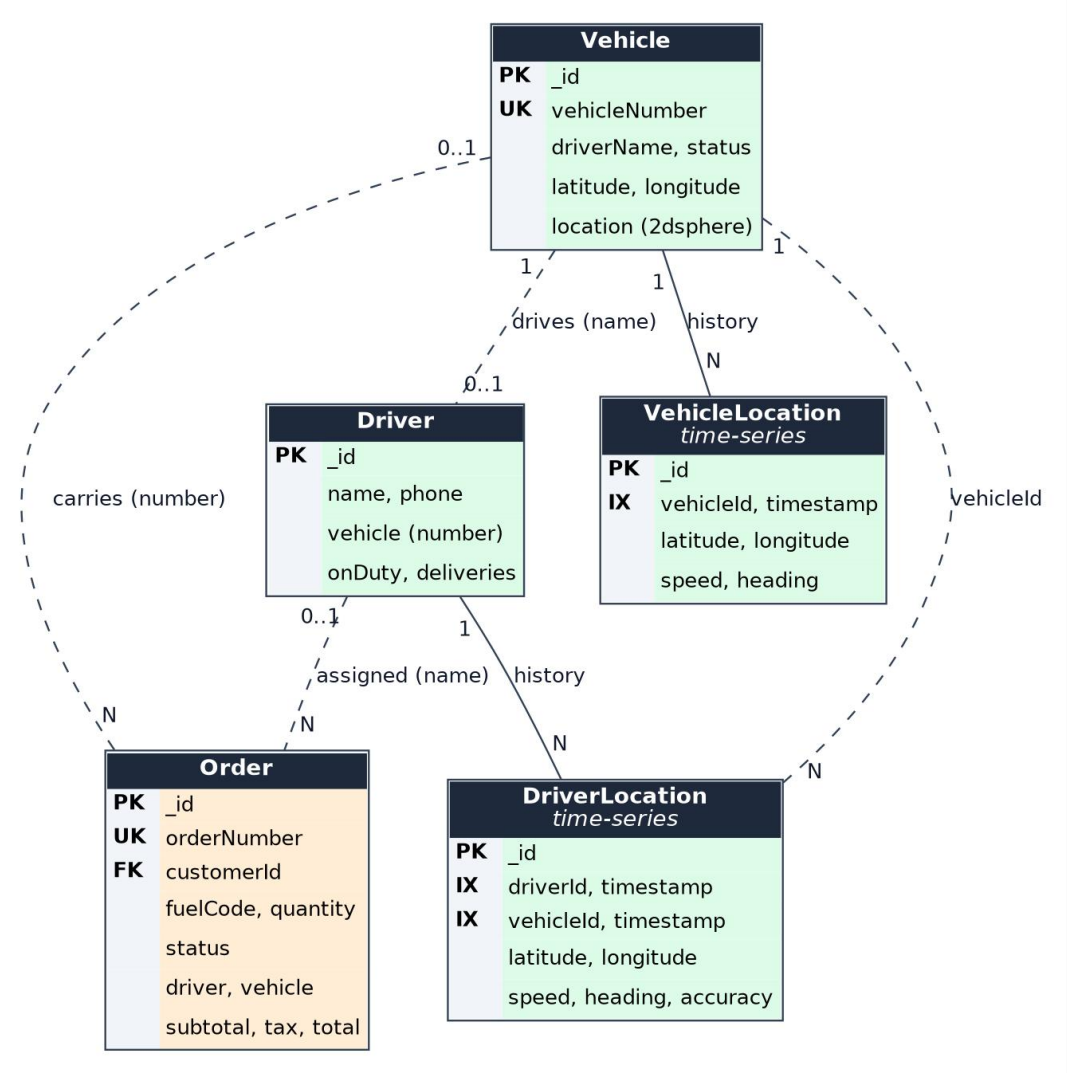


Figure 6.2: Fleet and Telemetry Entities Linkage Diagram

A critical architectural pattern utilized throughout the platform is immutable historical snapshotting. When an order is approved and dispatched, the assigned driver's name, vehicle plate number, fuel price, tax rate, and delivery freight are denormalized and stored directly on the Order document. Consequently, if a driver resigns, a vehicle undergoes re-registration, or state petroleum tariffs fluctuate in the future, completed historical orders remain completely unaffected, maintaining strict legal and financial auditability.

CHAPTER 7
TECH STACK

7.1 TECH STACK

The Smart Fuel Dispatch & Fleet Telemetry System is built entirely on modern web engineering standards, utilizing a cohesive JavaScript/Node.js full-stack ecosystem. Every technology choice in the project has been strictly verified against the active source code, package manifests (`package.json`), and build configurations. The application integrates specialized frontend libraries for interactive mapping and reactive state management, an enterprise Express.js backend with WebSocket support, and a scalable cloud database deployment. Table 7.1 consolidates the overall project technology stack, detailing the exact role and responsibility of each component.

Layer	Technology	Role & Operational Purpose
Frontend Core	React 18.2 with Vite 5.1	Single-page application framework delivering reactive component rendering and fast HMR development.
Client Routing	React Router DOM 6.22	Declarative client-side routing, nested route layouts, and role-based protected route wrappers.
State Management	React Context API & useReducer	Global state orchestration across eight dedicated contexts (Auth, Orders, Inventory, Notification, etc.).
HTTP Client	Axios 1.20	Promise-based REST client configured with automatic Bearer token injection and global error handling.
Real-Time Gateway	Socket.IO 4.8 (Client & Server)	Bi-directional WebSocket streaming for instant GPS coordinates, dispatch alerts, and notifications.
GIS & Mapping	Leaflet 1.9 & React-Leaflet 4.2	Interactive fleet visualization, coordinate markers, vehicle popup cards, and OpenStreetMap tiles.
Social Auth	@react-oauth/google & jwt-decode	Google OAuth 2.0 Identity Services client button integration and client-side JWT claims parsing.
UI Iconography	Lucide React 0.383	Lightweight SVG icon library integrated across all operational navigation and dashboard cards.
Backend Runtime	Node.js 20+ with Express 5.2	Asynchronous JavaScript server runtime providing high-throughput REST APIs and JSON endpoints.
Database Layer	MongoDB Atlas with Mongoose 9.9	Scalable cloud document store; ODM schemas, compound indexes, virtual getters, and lifecycle hooks.

Layer	Technology	Role & Operational Purpose
Authentication	jsonwebtoken 9.0 & bcryptjs 3.0	Stateless session authorization via signed JWTs and one-way cryptographic password salting and hashing.
OAuth Verification	google-auth-library 11.0	Cryptographic verification of Google OAuth 2.0 ID tokens against Google public RSA signing keys.
File Processing	Multer 2.3	Multipart/form-data upload middleware processing PDF, DOC, DOCX, JPEG, PNG KYC documents up to 10 MB.
Configuration & CORS	dotenv 17.4 & cors 2.8	Environment configuration isolation and cross-origin resource sharing policy management.
CI/CD Automation	Jenkins (Jenkinsfile)	Declarative build pipeline automating git checkout, npm installation, linting, build, and deploy stages.
Cloud Hosting	Vercel Platform (vercel.json)	Serverless deployment configuration routing static frontend assets and proxying `/api` to Express backend.

7.2 ARCHITECTURE OVERVIEW

The architecture of the Smart Fuel Dispatch & Fleet Telemetry System is structured following the classic Three-Tier / Model-View-Controller (MVC) architectural design, augmented with a real-time event-driven WebSocket gateway. This architectural separation ensures decoupling between presentation concerns, business rules, and persistence logic. Figure 7.1 illustrates the overall architectural flow and request-response lifecycle of the application.

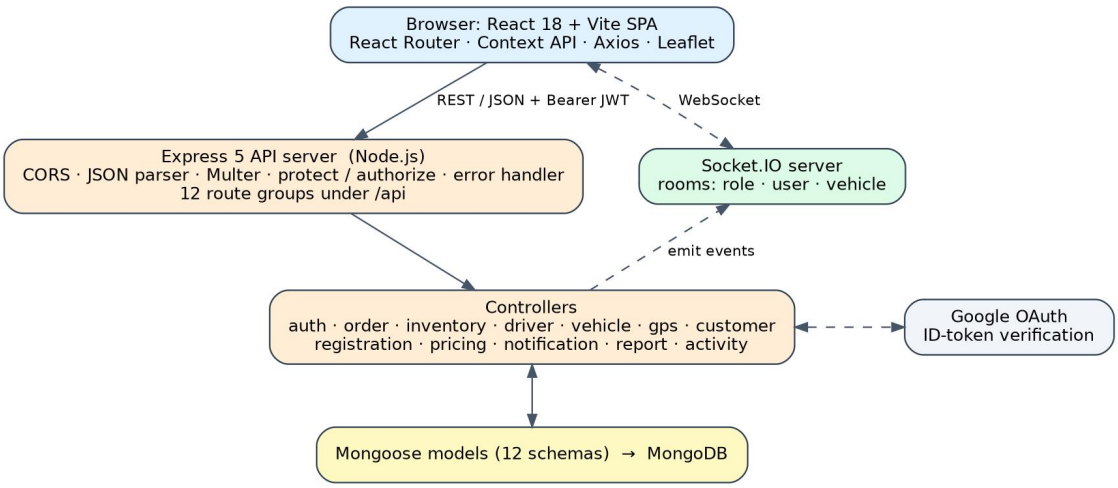


Figure 7.1: Overall Three-Tier MVC Architecture and Request Flow

As illustrated in Figure 7.1, the architectural layers interact across well-defined boundaries:

Presentation Tier (View): Implemented as a React SPA running in the user's browser. The UI layer consists of reusable interface components (Navbar, Sidebar, StatCards, ModalDialogs), role-specific page views (CustomerPortal, DriverPortal, Dashboard, FleetMap), and React Context providers. Axios dispatches asynchronous HTTPS requests to REST endpoints, while the Socket.IO client maintains a persistent WebSocket connection to receive real-time telemetry and alerts.

Application Tier (Controller): Hosted on the Node.js/Express server. HTTP requests pass through security middleware (CORS, body parser, and token authentication). Specialized Express controllers (`authController`, `orderController`, `inventoryController`, `vehicleController`) execute business validation, calculate Haversine proximity, reserve depot stock, and write audit trails. The Socket.IO server pushes room-targeted telemetry packets upon state updates.

Persistence Tier (Model): Managed by MongoDB Atlas through Mongoose schemas. Collections store user accounts, orders, tankers, drivers, vaults, and location history. Mongoose provides schema validation, compound indexes for fast lookups, and virtual fields for dynamic computations.

Architectural Layer	Implementation Components	Key Operational Responsibilities
Model (Data Tier)	Mongoose schemas, MongoDB Atlas	Encapsulates data models, schema validation rules, compound indexes, virtual getters, and database persistence.
View (Presentation)	React 18, Vite, Context API, Leaflet	Renders responsive user interfaces, maintains client state, captures user interactions, and renders real-time maps.
Controller (App Tier)	Express routes, controllers, middleware	Handles REST endpoints, enforces JWT authentication, executes business rules, and updates models.
Real-Time Gateway	Socket.IO server, WebSocket rooms	Broadcasts GPS coordinate pings, vehicle status updates, and critical system alerts to connected clients.

7.3 PROJECT SIZE & SOURCE CODE METRICS

The project represents a comprehensive full-stack codebase engineered to production standards. A quantitative audit of the repository source code verifies the extensive scope of implementation, summarized in Table 7.3.

Project Metric	Verified Count	Source Code Verification Basis
Frontend Pages	22 pages	React page components located in src/pages directory.
Reusable UI Components	14 components	Modular React components located in src/components.

Project Metric	Verified Count	Source Code Verification Basis
Global Context Providers	8 contexts	Auth, Orders, Inventory, Notification, Theme, Language, Settings, Simulation.
Frontend Services	3 services	REST api.js, WebSocket socket.js, and recentAccess.js cache.
Backend Route Gateways	13 route files	Mounted API routers in backend/routes handling 60+ endpoints.
Backend Controllers	13 controllers	Business logic modules located in backend/controllers.
Database Schemas	12 core schemas	15 model files in backend/models (Tank, Customer are aliases).
User Stakeholder Roles	6 roles	Admin, Depot Manager, Customer, Driver, Support Executive, Auditor.
Supported Languages	3 languages	English, Tamil, and Hindi translation dictionaries in src/locales.
Automated Test Suites	9 test scripts	Dedicated verification scripts in backend/ validating all flows.

CHAPTER 8
MODULES

8.0 DESIGN APPROACH

The Smart Fuel Dispatch & Fleet Telemetry System is partitioned into domain-specific operational modules following high-cohesion, low-coupling design principles. Each module encapsulates an independent operational facet of the bulk petroleum supply chain—from identity verification and customer onboarding to physical tanker dispatch, live telemetry ingestion, underground vault monitoring, and administrative reporting.

To ensure architectural consistency across the entire documentation, each operational module is evaluated against standard software engineering criteria: operational purpose and functional scope, non-functional constraints, workload scaling estimates, interface design (API endpoints), and database storage mapping. In strict accordance with report guidelines, detailed Low-Level Architecture (LLA) specifications and low-level component diagrams are focused exclusively on the first two foundational modules: Authentication & Access Control (Section 8.2) and Order & Dispatch Management (Section 8.3). The overall technology stack is presented once in Chapter 7 and is not repeated redundantly within each individual module.

Engineering Criterion	Key Focus & Analytical Content
Purpose & Scope	Defines the core operational responsibilities, target stakeholders, and business boundaries of the module.
Functional Requirements	Enumerates the verifiable features, workflows, validation checks, and operational transitions.
Non-Functional Requirements	Specifies system quality attributes, including security, latency, data integrity, and availability.
Scale & Workload Estimation	Back-of-the-envelope quantitative sizing assessing daily transactions, memory, and storage growth.
Low-Level Architecture (Modules 1 & 2)	Component structure, internal functions, data flow, and low-level architecture diagram.
API Interface Design	RESTful HTTP endpoints, supported request methods, payload structures, and authorization guards.
Database Schema Design	MongoDB collections, indexed fields, document references, and persistent state representations.
Operational Interfaces & Evidence	Genuine application interface screenshots, annotated features, and user interaction walkthroughs.

Shared Architectural Baseline: All modules adhere to a common architectural baseline. Every incoming HTTP request from the React client passes through Express security layers, where CORS validates origin headers, Express JSON middleware parses request payloads, and the `protect` and `authorize` middleware verify JWT signatures and RBAC roles. Database transactions operate on MongoDB Atlas, ensuring consistent document reads and writes. Passwords are never stored in plaintext and are salted and hashed using bcryptjs. The sections that follow present the complete module specifications.

8.1 FOLDER STRUCTURE

The project repository is strictly partitioned into two decoupled workspaces: a client-side React SPA workspace and a server-side Node.js/Express API workspace. This decoupled structure allows independent scaling, testing, and continuous deployment. The folder structure diagrams below illustrate the organization of the codebase.

Project Root Layout

At the root directory level, configuration files govern dependencies, environment variables, build scripts, and deployment automation pipelines. Figure 8.1 illustrates the project root organization.

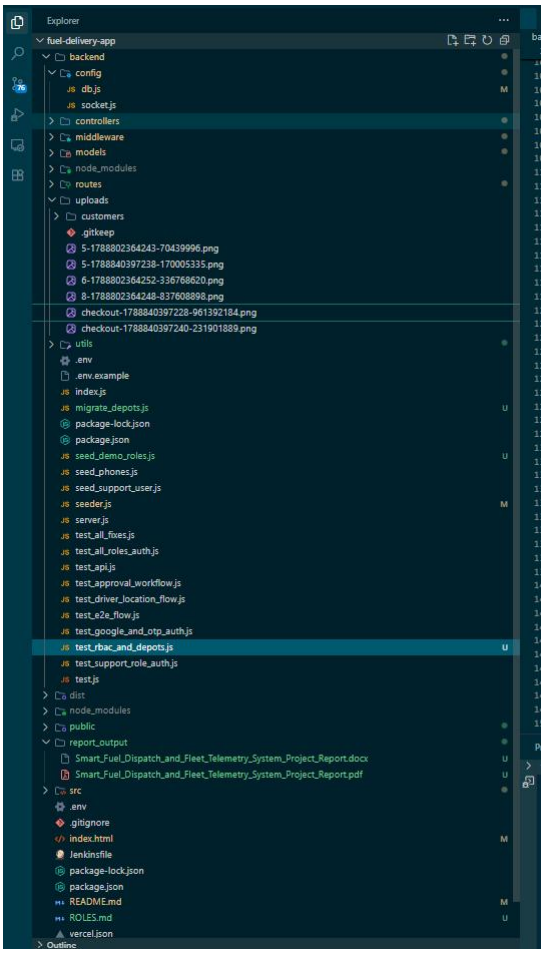


Figure 8.1: High-Level Project Root Directory Structure

Key root files include `package.json` and `package-lock.json` defining frontend dependencies, `vite.config.js` configuring build optimizations and development proxies, `index.html` acting as the single-page entry point, `Jenkinsfile` declaring the CI/CD stages, and `vercel.json` managing multi-service cloud hosting.

Backend Architecture and Folder Structure

The backend server is entirely self-contained within the `backend/` directory. It adheres strictly to the Model-View-Controller design pattern, decoupling routing definitions from business controllers and database schemas. Figure 8.2 illustrates the backend directory layout.

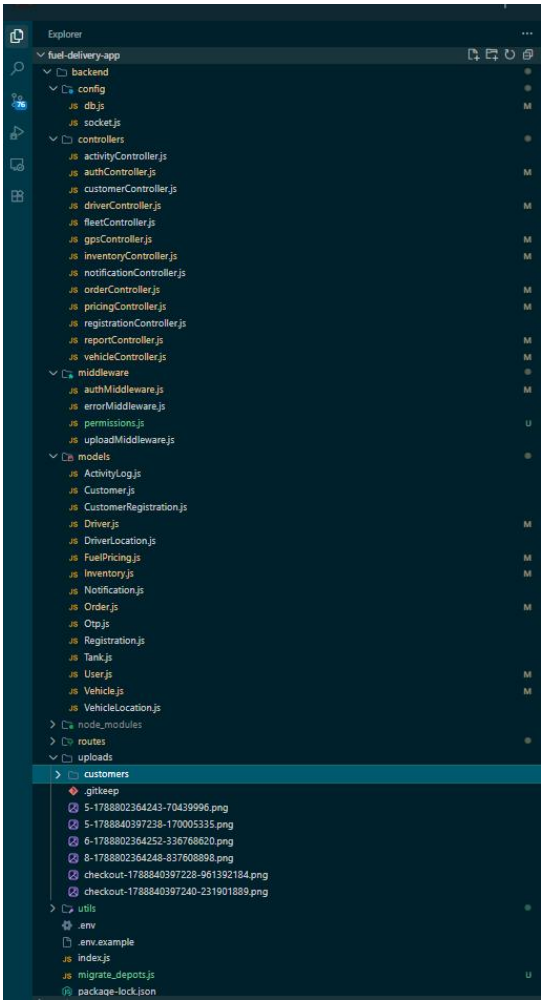


Figure 8.2: Backend Architecture and Directory Layout

The backend structure contains seven core functional subdirectories: `config/` (database connection and environment setup), `controllers/` (business logic for authentication, orders, inventory, fleet, and reports), `middleware/` (JWT token verification and role authorization guards in `permissions.js`), `models/` (15 Mongoose schema definitions), `routes/` (13 mounted REST route modules), `uploads/` (secure local storage for customer KYC documents), and `utils/` (helper utilities).

Frontend Architecture and Folder Structure

The frontend is organized within the `src/` directory, following a feature-oriented component architecture. Figure 8.3 illustrates the frontend source organization.

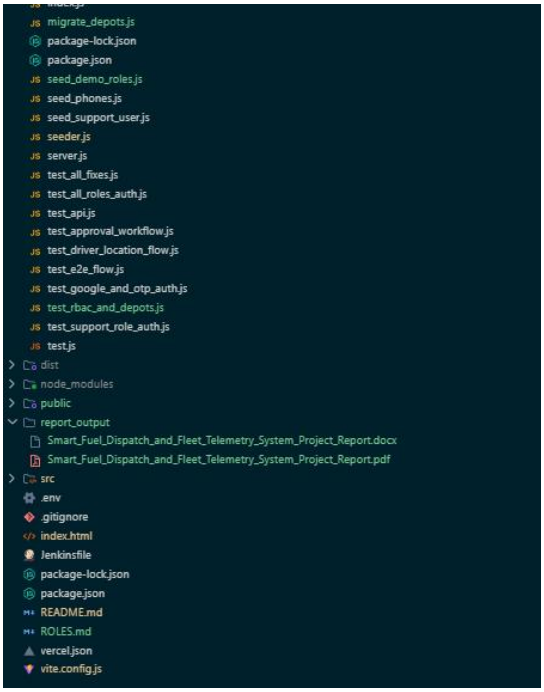


Figure 8.3: Frontend Application Directory Layout (src/)

The frontend source is divided into: `components/` (14 reusable UI components like Navbar, Sidebar, StatCard, and Leaflet markers), `contexts/` (8 global React Context providers orchestrating authentication, orders, inventory, and notifications), `pages/` (22 specialized screen views for all stakeholder roles), `services/` (Axios API client and Socket.IO connection handlers), and `locales/` (translation dictionaries supporting English, Tamil, and Hindi).

Client-Server Interaction Boundary

The frontend React client never communicates directly with the MongoDB database. Every data operation is mediated via authenticated HTTP REST calls through Axios or real-time event packets via Socket.IO. The client attaches the JWT Bearer token to request headers, and the backend verifies permissions before executing database queries.

8.2 AUTHENTICATION & ACCESS CONTROL

The Authentication & Access Control module is the foundational security gateway of the Smart Fuel Dispatch & Fleet Telemetry System. It governs identity verification, multi-factor authentication, session management, and Role-Based Access Control (RBAC) across six user roles: Administrator, Depot Manager, Customer, Driver, Support Executive, and Auditor.

Purpose and Operational Responsibilities

The module ensures that only authenticated and authorized users access platform resources. It supports three distinct login modes: standard email and password, Google OAuth 2.0 single sign-on, and mobile phone OTP verification. Crucially, customer accounts in 'Pending Approval' or 'Rejected' states are strictly barred from logging in, ensuring that unverified businesses cannot place fuel orders or view pricing.

Functional Requirements

Multi-Modal Sign-In: Supports email/password authentication, Google OAuth 2.0 social sign-in, and 6-digit SMS-ready OTP login.

Password Security: Hashes all user passwords using bcryptjs with 10 salt rounds; plaintext passwords are never stored.

JWT Token Issuance: Signs cryptographically secure JWT tokens containing user ID, email, and role, configured with a 30-day validity window.

Account Verification Gate: Blocks login attempts from corporate customer accounts that have not received explicit administrative approval.

Role-Based Route Guards: Enforces client-side navigation protection via `ProtectedRoute.jsx` and server-side API protection via `permissions.js`.

Non-Functional Requirements and Security Constraints

Security: Passwords must withstand offline dictionary and rainbow table attacks via bcryptjs salting. Session tokens must be stateless, tamper-proof, and validated on every API call. Latency: Authentication verification, password hashing, and token generation must execute within 200 milliseconds. Availability: Authentication endpoints must maintain 99.9% uptime to prevent operational dispatch bottlenecks.

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Active User Base	250 accounts	Fleet drivers, depot personnel, customers, and corporate administrators.
Daily Login Operations	600 sessions / day	Average 2 to 3 logins per user per shift; peak logins during morning roster.
Password Verification Load	0.01 ops / sec avg	Handled effortlessly by single Node.js event loop with bcryptjs worker threads.
JWT Token Verifications	15,000 req / day	Every authenticated REST API request triggers synchronous cryptographic token verification.
Token Invalidation Overhead	Negligible	Stateless JWT verification eliminates server-side session lookup latency.

Detailed Low-Level Architecture (Module 1)

The internal component structure of the Authentication module spans both frontend React architecture and backend Express modules:

Frontend Components: `Login.jsx` (presents tabbed authentication interfaces for password, Google OAuth, and OTP), `Register.jsx` (handles user registration), `ForgotPassword.jsx` (initiates password reset), and `AuthContext.jsx` (maintains reactive user state, token persistence in localStorage, and logout teardown).

Backend Middleware & Controllers: `authController.js` (contains `loginUser`, `registerUser`, `verifyOTP`, `googleAuth`, `getMe`, and `updateProfile`), `permissions.js` (middleware providing `protect` token verification and `authorize(...roles)` RBAC filtering), and `User.js` / `Otp.js` (Mongoose schema models).

Figure 8.4 illustrates the Low-Level Architecture diagram for the Authentication & Access Control module, detailing the exact data flow between the user interface, Express middleware, Google OAuth servers, and MongoDB Atlas.

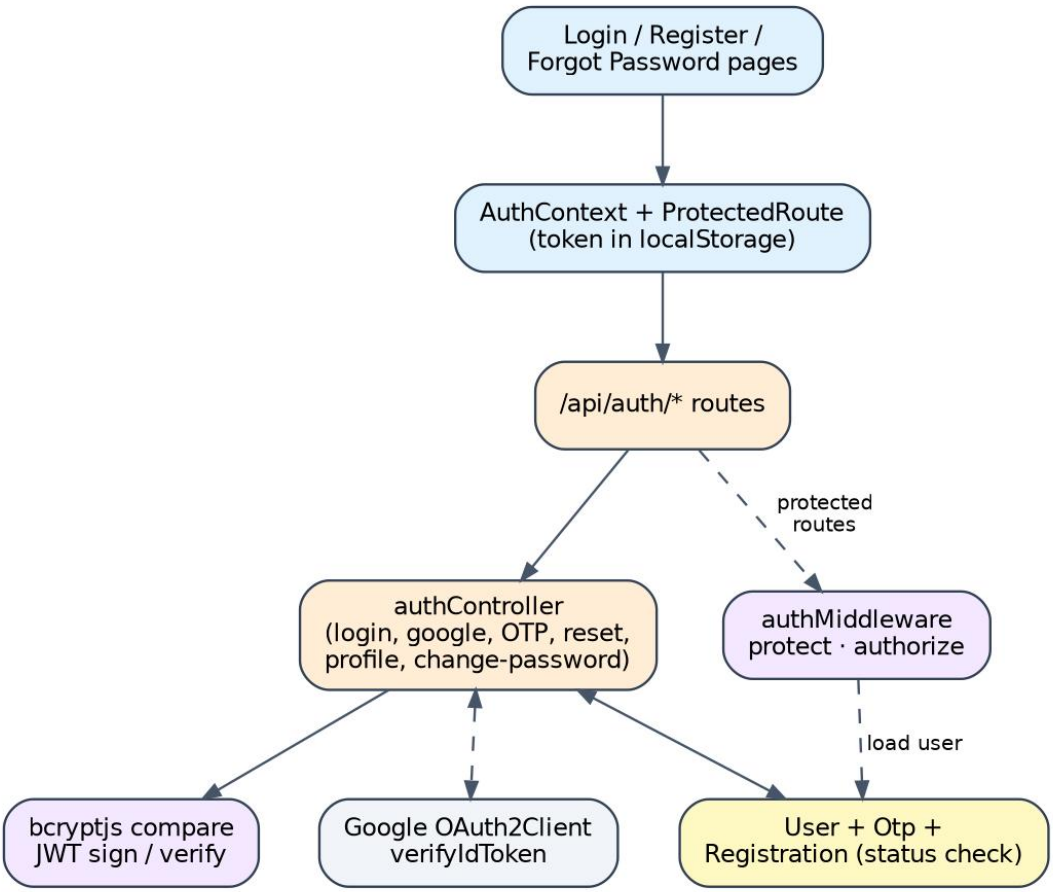


Figure 8.4: Low-Level Architecture Diagram – Authentication & Access Control Module

Explanation of Low-Level Flow: When a user submits credentials in `Login.jsx`, Axios dispatches an HTTPS POST request to `/api/auth/login`. In `authController.js`, `loginUser` queries the `users` collection in MongoDB Atlas by email. If the user exists, `bcrypt.compare()` validates the plaintext password against the stored bcrypt hash. Next, the controller checks `user.isApproved` and `user.status`; if the account is a Customer that is pending or rejected, an HTTP 403 Forbidden response is immediately returned. Upon successful validation, `jwt.sign()` generates a signed JWT token containing `{ id, role }`. The response payload returns the token and sanitized user profile to `AuthContext.jsx`, which saves the token in `localStorage`, sets Axios default authorization headers, and redirects the user to their role-specific dashboard.

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
POST	/api/auth/login	Public	Authenticates user with email and password; returns JWT token and profile.
POST	/api/auth/register	Public	Registers new user account with default role Customer.
POST	/api/auth/google	Public	Verifies Google OAuth 2.0 credential token; provisions or signs in user.
POST	/api/auth/send-otp	Public	Generates 6-digit OTP code and records in MongoDB with 5-minute TTL.
POST	/api/auth/verify-otp	Public	Validates mobile OTP code; issues JWT token upon successful match.
GET	/api/auth/me	All Roles (Auth)	Fetches profile of currently authenticated user using Bearer token.
PUT	/api/auth/profile	All Roles (Auth)	Updates display name, phone, or company profile parameters.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy
users	Lookups by email, googleId, and _id	Unique index on email; index on googleId; index on role.
otps	Fast lookup by mobile number	Index on mobile; TTL index on createdAt with 300 second auto-expiry.

Operational Interface and Evidence

Figure 8.5 showcases the actual implemented Login Interface of the platform. The screen provides a clean, accessible user experience supporting standard credential input, Google OAuth one-click authentication, and mobile OTP verification.

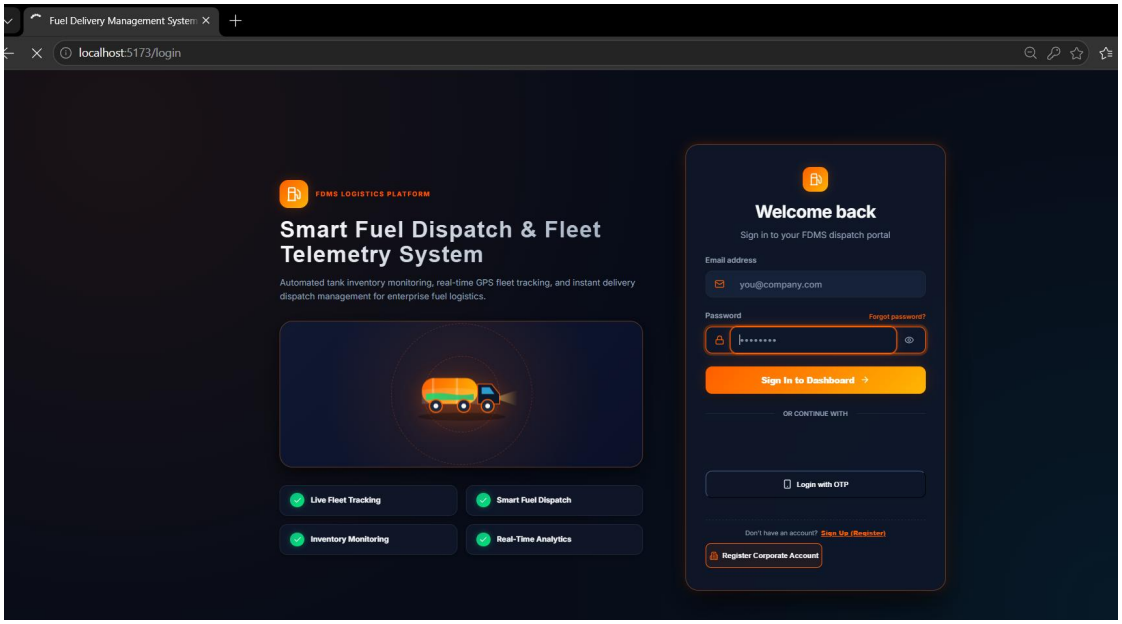


Figure 8.5: User Login Interface with Multi-Modal Authentication Options

The interface includes real-time input validation, responsive visual feedback, password visibility toggles, and clear links for corporate account registration and password recovery. Role-based redirect logic immediately takes Administrators to the Executive Dashboard, Depot Managers to the Approval Center, Drivers to the Driver Duty Portal, and Customers to the Order Portal.

8.3 ORDER & DISPATCH MANAGEMENT

The Order & Dispatch Management module is the commercial and operational core of the platform. It orchestrates the complete lifecycle of bulk petroleum transactions—from customer order placement and dynamic pricing to depot managerial approvals, automated inventory reservations, intelligent tanker/driver dispatch, and delivery proof generation.

Purpose and Operational Responsibilities

The module bridges corporate fuel demand with physical depot supply and fleet capacity. When an order is created, it calculates real-time tariffs, enforces minimum volume thresholds (100 L), and places the order in a pending queue. Depot Managers review pending requests, inspect real-time vault capacity, and approve or reject requests. Approving an order atomically reserves stock in the underground tank to prevent overselling. The module also features a Smart Dispatch algorithm that evaluates tanker distance and road proximity using the Haversine formula to recommend the optimal tanker.

Functional Requirements

Order Creation & Pricing: Enables verified corporate customers to place orders; computes base price, 18% GST, and freight charges.

Managerial Review & Approval: Provides an Approval Center for Depot Managers to inspect customer requests and approve or reject orders.

Atomic Stock Reservation: Atomically increments `reservedStock` in the depot tank when an order is approved, locking stock.

Smart Dispatch Recommendation: Calculates Euclidean distances to available tankers using Haversine equations, recommending the nearest tanker.

Driver & Tanker Assignment: Assigns an available tanker and driver, transitioning the order to 'Assigned' and 'Dispatched' states.

Delivery Completion & Stock Deduction: Marks deliveries complete upon driver confirmation, deducts vault stock, and generates digital invoices.

Non-Functional Requirements and Operational Constraints

Consistency & Concurrency: Fuel stock reservation must be atomic to eliminate race conditions between simultaneous orders. Accuracy: Financial computations (fuel amount, 18% GST, delivery fee) must maintain exact double-precision arithmetic. Throughput: Order querying, filtering, and status transitions must execute under 150 milliseconds.

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Daily Fuel Orders	80 - 150 orders / day	Corporate fleets, manufacturing plants, and regional fuel stations.
Average Order Volume	3,500 Liters / order	Bulk diesel and petrol shipments ranging from 500 L to 15,000 L.
Total Daily Fuel Flow	~450,000 Liters / day	Aggregated throughput across regional depot distribution tanks.
Order Document Storage	~1.2 MB / day	BSON document footprint with delivery addresses, timestamps, and audit notes.
Annual Data Footprint	~450 MB / year	Extremely lightweight; easily maintained within MongoDB Atlas standard tiers.

Detailed Low-Level Architecture (Module 2)

The internal component structure of the Order & Dispatch Management module comprises the following components:

Frontend Components: `Orders.jsx` (order management register with multi-criteria filters, search, and CSV export), `PendingRequests.jsx` (Depot Manager approval queue), `OrderDetail.jsx` (detailed order invoice and dispatch timeline), and `OrderContext.jsx` (manages global order state via useReducer).

Backend Controllers & Models: `orderController.js` (executes `createOrder`, `approveOrder`, `rejectOrder`, `assignDriverTanker`, `updateOrderStatus`, `getSmartDispatchRecommendation`, and `completeDelivery`), `Order.js` (core Mongoose schema), `pricingController.js` (fuel price lookups), and `inventoryController.js` (tank stock reservation and deduction).

Figure 8.6 illustrates the Low-Level Architecture diagram for the Order & Dispatch Management module, depicting the complete data flow from order submission to depot stock deduction and driver dispatch.

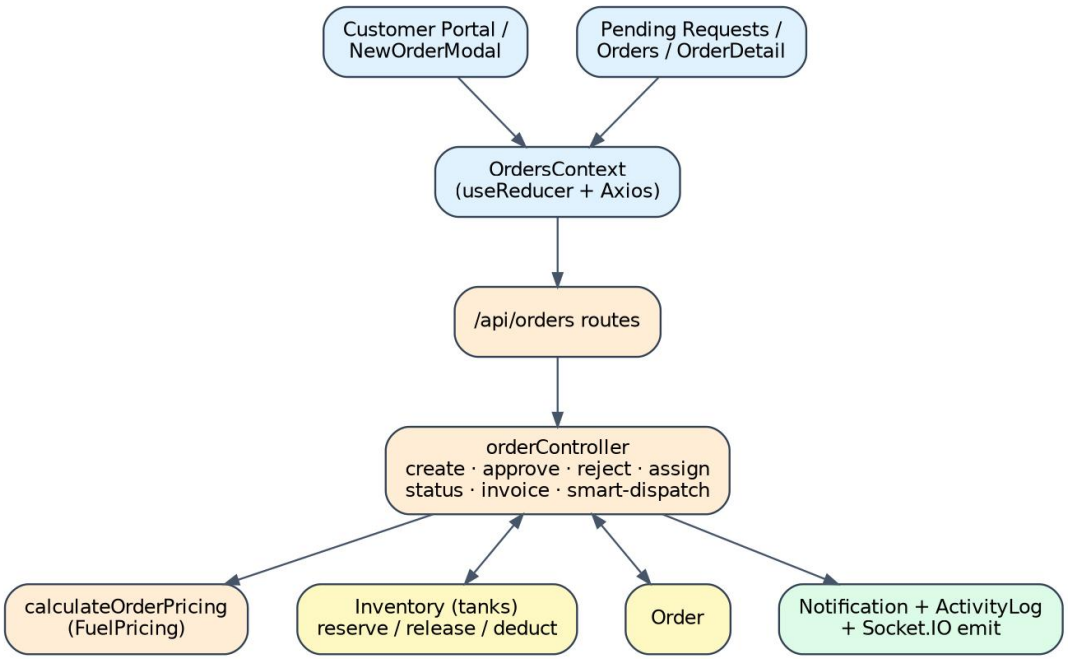


Figure 8.6: Low-Level Architecture Diagram – Order & Dispatch Management Module

Explanation of Low-Level Flow: When an order is placed, `orderController.js` fetches active tariffs from `FuelPricing` to calculate base price, 18% GST, and freight. When the Depot Manager clicks 'Approve' in `PendingRequests.jsx`, an HTTP PUT request triggers `approveOrder`. The controller verifies that available stock (`currentStock - reservedStock`) in the matching `Tank` exceeds the requested volume. If verified, Mongoose atomically updates `reservedStock += quantity` on the tank and advances the order status to 'Approved'. When the manager invokes the Smart Dispatch endpoint, the system calculates the Haversine distance between the delivery coordinates and all active `Vehicle` positions, recommending the nearest tanker. Assigning a driver and tanker transitions the order to 'Assigned'. When the driver confirms delivery, `completeDelivery` decrements both `currentStock` and `reservedStock` by the order volume, transitions the vehicle back to 'Available', updates driver trip counts, and emits a real-time completion alert via Socket.IO.

Figure 8.7 illustrates the complete Order Lifecycle state machine and its direct synchronization with underground storage vault stock.

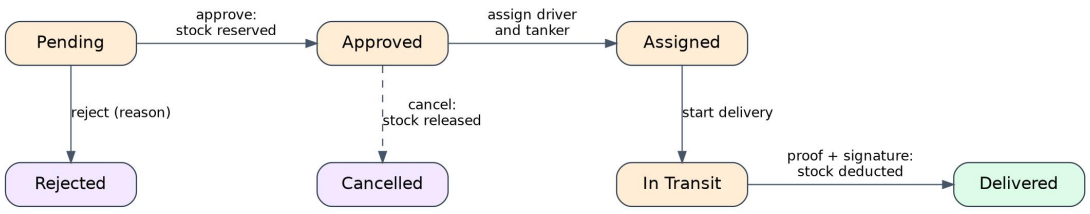


Figure 8.7: Fuel Order Lifecycle State Machine and Depot Tank Stock Impact

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
POST	/api/orders	Customer, Admin	Creates new fuel order with customer details, fuel type, volume, and site address.
GET	/api/orders	All Roles (Auth)	Retrieves order register; customers view own orders; managers view all orders.
GET	/api/orders/:id	All Roles (Auth)	Fetches detailed order specification, pricing breakdown, and dispatch status.
PUT	/api/orders/:id/approve	Depot Manager, Admin	Approves pending order and atomically reserves fuel stock in matching tank.
PUT	/api/orders/:id/reject	Depot Manager, Admin	Rejects order request with justification notes; unlocks reserved stock if any.
PUT	/api/orders/:id/assign	Depot Manager, Admin	Assigns tanker vehicle and driver; updates status to 'Assigned'.
PUT	/api/orders/:id/status	Depot Manager, Driver	Advances lifecycle status: Dispatched, In Transit, Delivered, Cancelled.
GET	/api/orders/:id/smart-dispatch	Depot Manager, Admin	Calculates Haversine distance and recommends nearest available tanker.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy
orders	Lookups by orderNumber, customerId, status, and createdAt	Unique index on orderNumber; compound indexes on (customerId, createdAt) and (status, createdAt).
tanks	Atomic updates for stock reservation and deduction	Index on fuelType; index on tankName.

Operational Interfaces and Evidence

Figure 8.8 illustrates the Depot Manager Approval Center (`PendingRequests.jsx`), where incoming customer orders are reviewed, verified against vault stock, and either approved or rejected with operational notes.

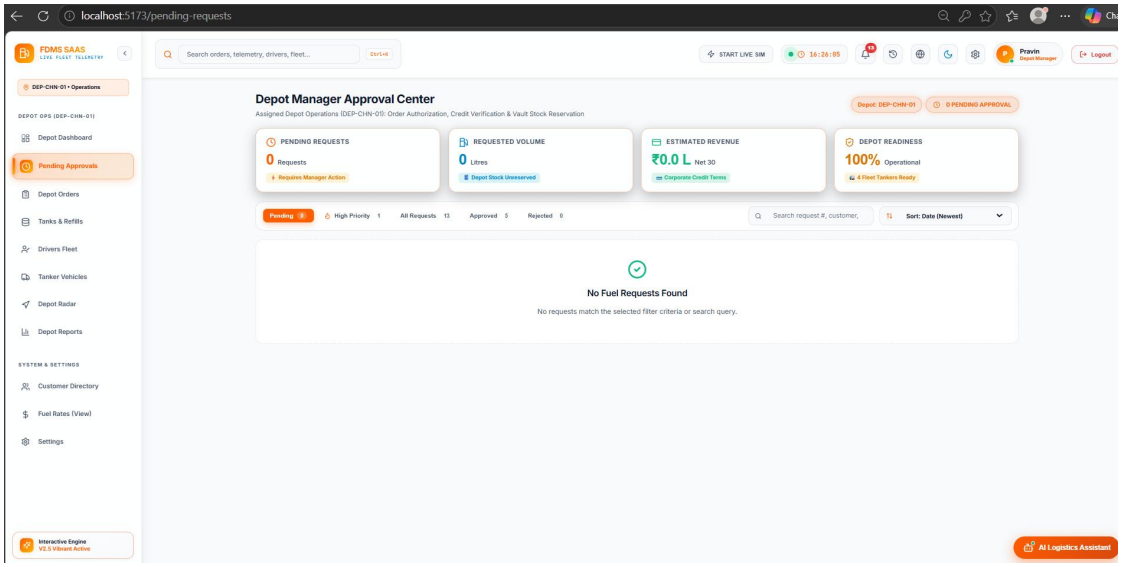


Figure 8.8: Depot Manager Approval Center (Pending Requests Management)

Figure 8.9 showcases the Orders Management page (`Orders.jsx`), providing comprehensive search, status filtering, CSV export functionality, and direct links to order details and dispatch tracking.

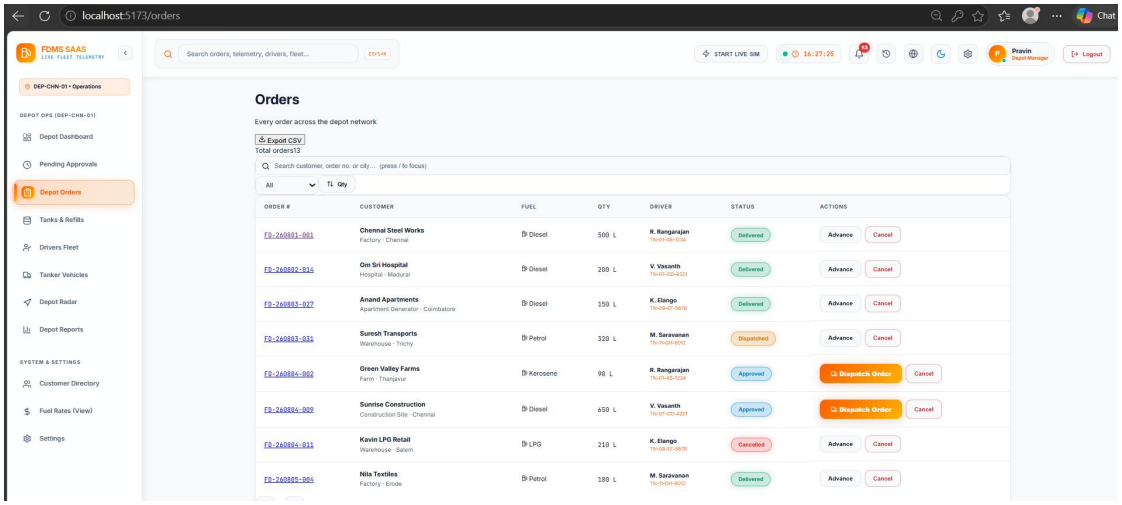


Figure 8.9: Orders Management Register with Multi-Criteria Search, Filters, and CSV Export

8.4 FLEET, DRIVER & GPS TELEMETRY

The Fleet, Driver & GPS Telemetry module provides real-time geospatial monitoring, asset management, and telemetry diagnostics for transport resources. It tracks mobile tankers and drivers across transit routes, capturing live GPS coordinates, vehicle diagnostic parameters, and duty status.

Purpose and Scope

The module ingests continuous coordinate streams from mobile GPS modems or driver mobile devices via REST and WebSockets. It renders live vehicle positions on an interactive Leaflet map of Tamil Nadu, draws historical breadcrumb trails (up to 1,000 points per

vehicle), and monitors critical diagnostic parameters including fuel level, tyre pressure, and engine temperature.

Functional Requirements

Real-Time GPS Map: Renders tanker locations dynamically on an interactive map using custom colored icons reflecting vehicle status.

Telemetry Breadcrumbs: Stores chronological historical location points in `VehicleLocation`, allowing route replay and audit.

Driver Workforce Management: Maintains driver profiles, duty status toggles (Available, On Duty, Inactive), and trip counters.

Tanker Diagnostic Tracking: Monitors onboard tank fill percentage, diagnostic tyre pressure (PSI), and engine coolant temperature (°C).

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Active Tanker Fleet	25 vehicles	Fleet of heavy commercial fuel tankers operating across regional depot corridors.
Telemetry Ping Frequency	Every 15 - 30 seconds	Periodic GPS updates transmitted during transit operations.
Daily Location Pings	~40,000 pings / day	Aggregated coordinate packets ingested via REST and Socket.IO.
Historical Data Storage	~4.8 MB / day	Append-only BSON telemetry documents stored in VehicleLocation collection.

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
GET	/api/vehicles	All Roles (Auth)	Retrieves tanker vehicles register with diagnostic and telemetry status.
POST	/api/vehicles	Depot Manager, Admin	Registers new tanker vehicle with plate number, cargo type, and capacity.
POST	/api/vehicles/location	Driver, System	Ingests GPS telemetry ping (lat, lng, speed, heading, vehicleId).
GET	/api/vehicles/:id/history	All Roles (Auth)	Fetches chronological GPS breadcrumb history for route visualization.
GET	/api/drivers	All Roles (Auth)	Retrieves driver workforce roster, duty status, and assigned vehicles.
POST	/api/drivers	Depot Manager, Admin	Onboards new driver with contact details, license number, and vehicle.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy
vehicles	Lookups by vehicleNumber and status	Unique index on vehicleNumber; index on status.
drivers	Lookups by phone, licenseNumber, and status	Unique index on phone; index on status.
VehicleLocation	High-frequency historical queries	Compound index on (vehicleId, timestamp descending).

Operational Interfaces and Evidence

Figure 8.10 showcases the Fleet GPS Control Center (`FleetMap.jsx`), featuring an interactive Leaflet map of Tamil Nadu with real-time tanker coordinate markers, route paths, and detailed vehicle status popup cards.

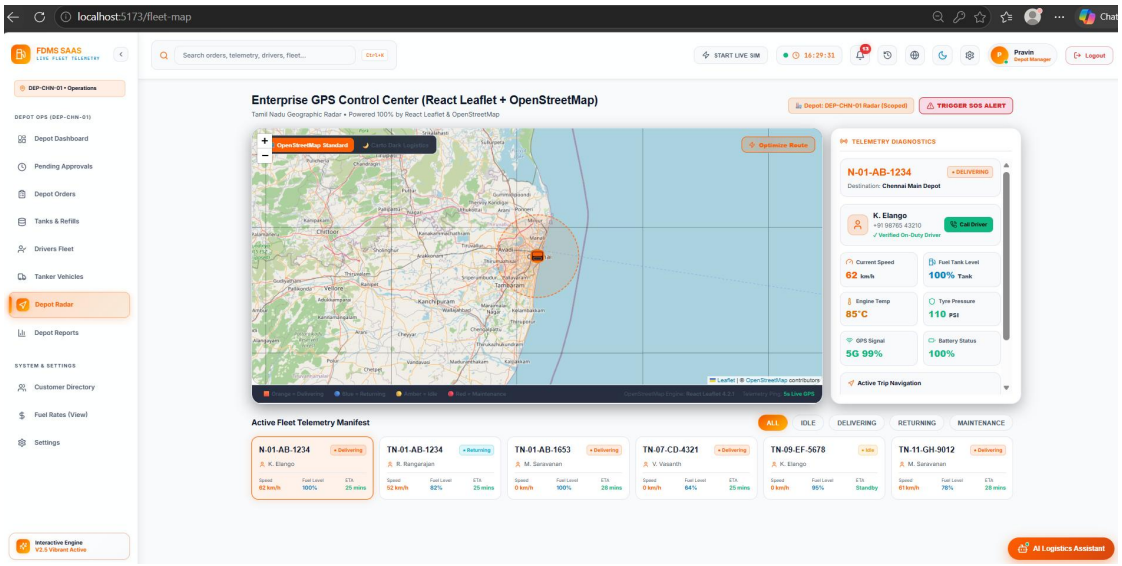


Figure 8.10: Fleet GPS Control Center with Interactive Real-Time Map and Telemetry Cards

Figure 8.11 illustrates the Driver Workforce and Duty Roster interface (`Drivers.jsx`), allowing managers to register drivers, assign vehicles, monitor completed trips, and adjust operational duty states.

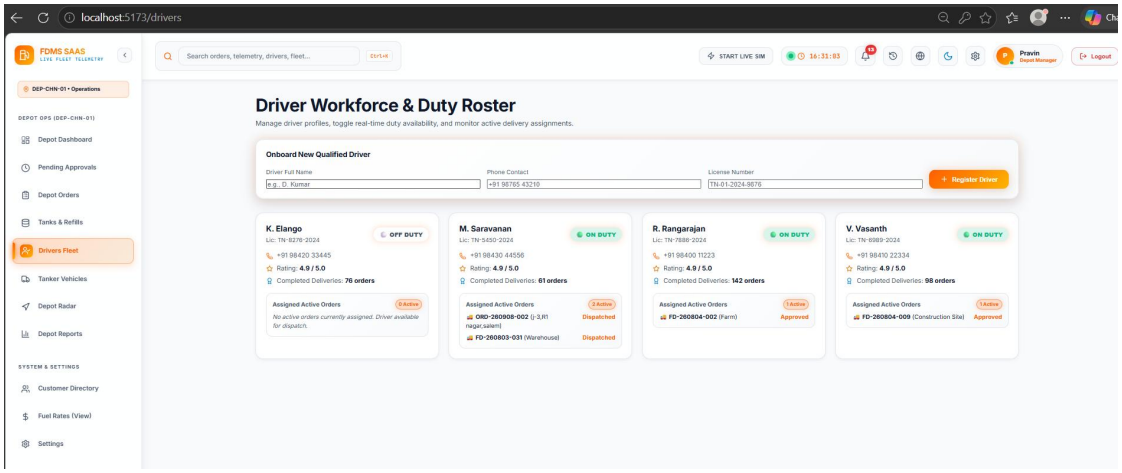


Figure 8.11: Driver Workforce and Duty Roster Management Interface

Figure 8.12 displays the Fleet Tanker Vehicles Management interface (`Vehicles.jsx`), summarizing asset availability, cargo specifications, and onboard telemetry parameters.

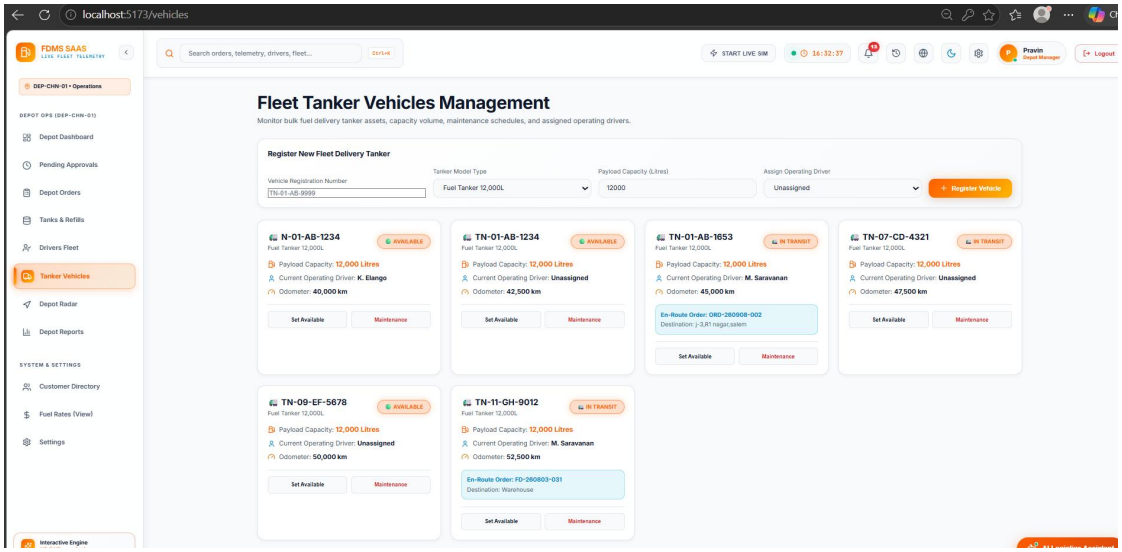


Figure 8.12: Fleet Tanker Vehicles Management Register and Status Controls

8.5 INVENTORY & FUEL PRICING

The Inventory & Fuel Pricing module maintains oversight of depot petroleum storage vaults and central commercial rates. It monitors underground vault capacity, tracks reserved stock allocations, alerts managers to low-stock conditions, and maintains transparent fuel pricing across all supported petroleum types.

Purpose and Scope

Depot vaults store thousands of liters of volatile hydrocarbons. The module prevents physical stock-outs by tracking both gross physical inventory and committed reservations in real time. Virtual schema getters calculate available stock and fill percentages, automatically triggering visual warnings when stock falls below configured safety thresholds. The pricing subsystem manages base prices per liter, standard tax percentages (18% GST), and delivery freight rates.

Functional Requirements

Vault Telemetry Tracking: Tracks physical volume, reserved volume, fill percentage, temperature (°C), and pressure (PSI).

Low-Stock Alarms: Automatically generates visual warning badges and alert notifications when stock drops below minimum threshold.

Vault Refill Operations: Allows depot managers to record bulk replenishment deliveries, updating current stock and refill history.

Dynamic Pricing Engine: Maintains real-time commercial prices per liter, GST percentages, and freight rates for Diesel, Petrol, and CNG.

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Monitored Depot Vaults	10 - 20 storage tanks	Regional underground storage tanks holding Diesel, Petrol, Premium Diesel, and CNG.
Stock Updates / Day	150 - 300 transactions	Triggered automatically upon order approvals, dispatches, and bulk replenishments.
Price Adjustments	1 - 2 updates / week	Commercial tariff revisions updated periodically by corporate administration.

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
GET	/api/inventory	All Roles (Auth)	Fetches depot vault list with capacity, current stock, reserved stock, and status.
POST	/api/inventory	Depot Manager, Admin	Creates new vault record with capacity, fuel type, and safety threshold.
PUT	/api/inventory/:id/refill	Depot Manager, Admin	Records bulk fuel replenishment; increments physical stock and logs refill.
GET	/api/pricing	All Roles (Auth)	Fetches active pricing tariffs for all fuel types including GST and delivery charge.
PUT	/api/pricing/:code	Admin	Updates tariff rates per liter, tax rate, or delivery fee for specified fuel code.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy

Collection Name	Access Pattern	Indexing Strategy
tanks	Lookups by fuelType and tankName	Index on fuelType; index on tankName.
fuelpricings	Lookups by fuel code	Unique index on code (DSL, PTL, PDSL, CNG).

Operational Interface and Evidence

Figure 8.13 showcases the Depot Fuel Vault Telemetry screen (`Inventory.jsx`), displaying storage tank cards with dynamic gauge bars, temperature and pressure telemetry, and interactive refill controls.

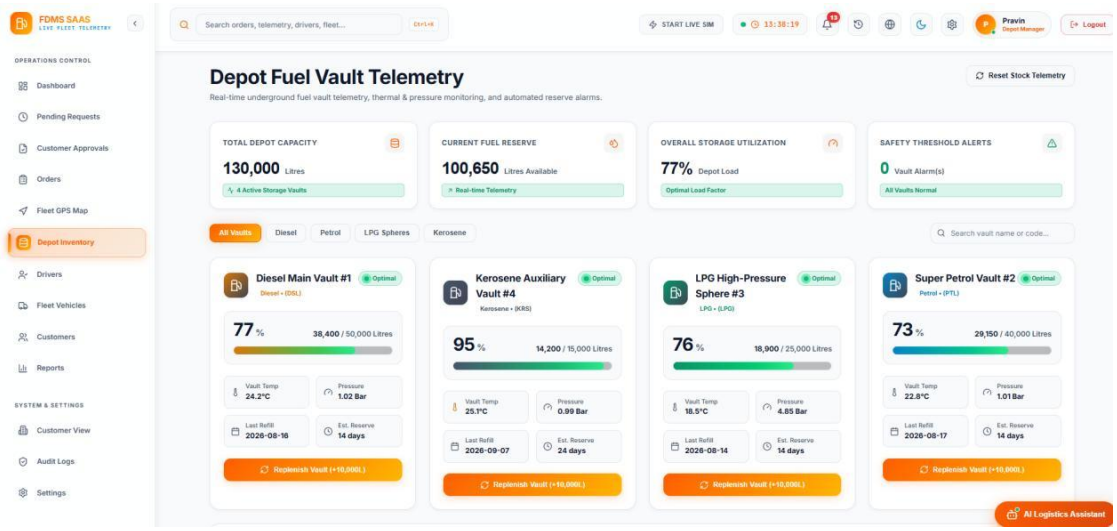


Figure 8.13: Depot Fuel Vault Telemetry and Storage Tanks Management Interface

8.6 CUSTOMER, REGISTRATION & DOCUMENT MANAGEMENT

The Customer, Registration & Document Management module implements corporate Know-Your-Customer (KYC) onboarding and corporate account administration. Because bulk fuel is a regulated commercial commodity, corporate buyers must undergo statutory verification before gaining order capabilities.

Purpose and Scope

The module provides a self-service registration portal where companies submit corporate identification, GSTIN, PAN, office addresses, contact coordinates, and projected fuel volumes. Applicants upload statutory verification documents (incorporation certificates, tax registrations) via multipart form uploads handled by Multer. Submissions are queued for administrative review. Approving an application automatically creates a verified Customer user account and dispatches an acceptance notification.

Functional Requirements

Corporate Registration Form: Captures company trade name, contact person, email, phone, GSTIN, PAN, and delivery address.

KYC Document Upload: Supports multipart file uploads (PDF, DOC, DOCX, PNG, JPEG) up to 10 MB per document using Multer.
Reviewer Approval Workflow: Allows Depot Managers and Admins to inspect applications, view uploaded files, and approve or reject submissions.
Customer Master Directory: Maintains an active directory of verified corporate clients with order history and compliance status.

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Onboarding Applications	5 - 15 applications / week	Commercial fleet operators, manufacturing plants, infrastructure builders.
Document Storage Growth	~15 - 50 MB / month	Statutory PDF/JPEG business documents stored in secure backend upload directory.
Verified Corporate Accounts	150+ active companies	Maintaining active enterprise fuel procurement accounts.

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
POST	/api/registrations	Public	Submits corporate registration application with multipart document uploads.
GET	/api/registrations	Depot Manager, Admin	Retrieves application queue filtered by status (Pending, Approved, Rejected).
PUT	/api/registrations/:id/status	Depot Manager, Admin	Approves or rejects application; provisions User account upon approval.
GET	/api/customers	All Roles (Auth)	Fetches verified corporate customers directory with order volume summaries.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy
customerregistrations	Lookups by regId, email, and status	Unique index on regId; index on email; index on status.

Operational Interfaces and Evidence

Figure 8.14 illustrates the Customer Applications and KYC Approvals portal (`Applications.jsx`), where managers inspect incoming corporate registrations, review uploaded tax documents, and approve accounts.

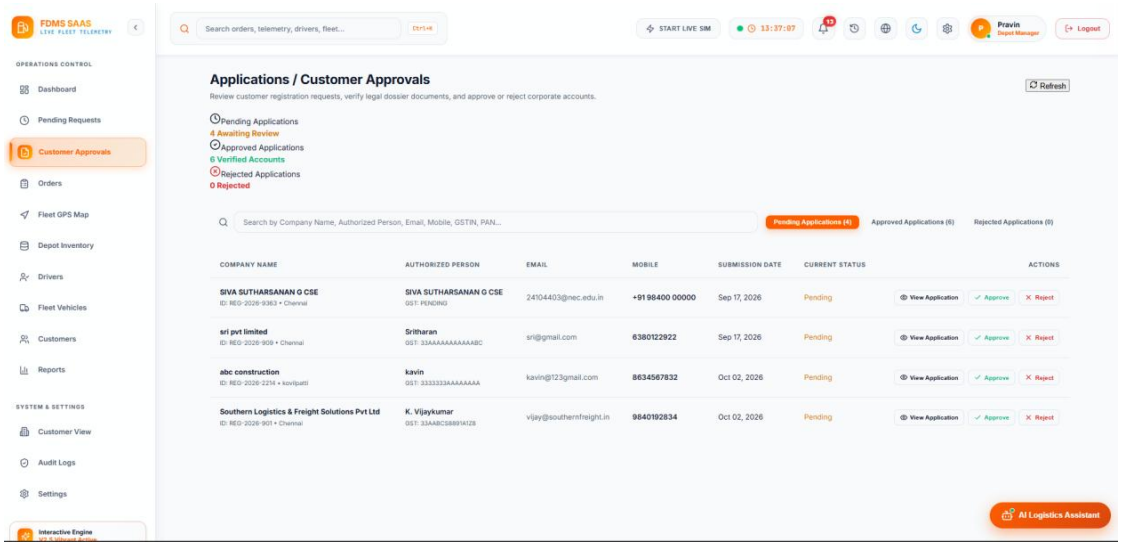


Figure 8.14: Customer Applications and KYC Review Approval Portal

Figure 8.15 displays the Corporate Customers Directory (`Customers.jsx`), summarizing active clients, contact information, registered fuel consumption, and verification statuses.

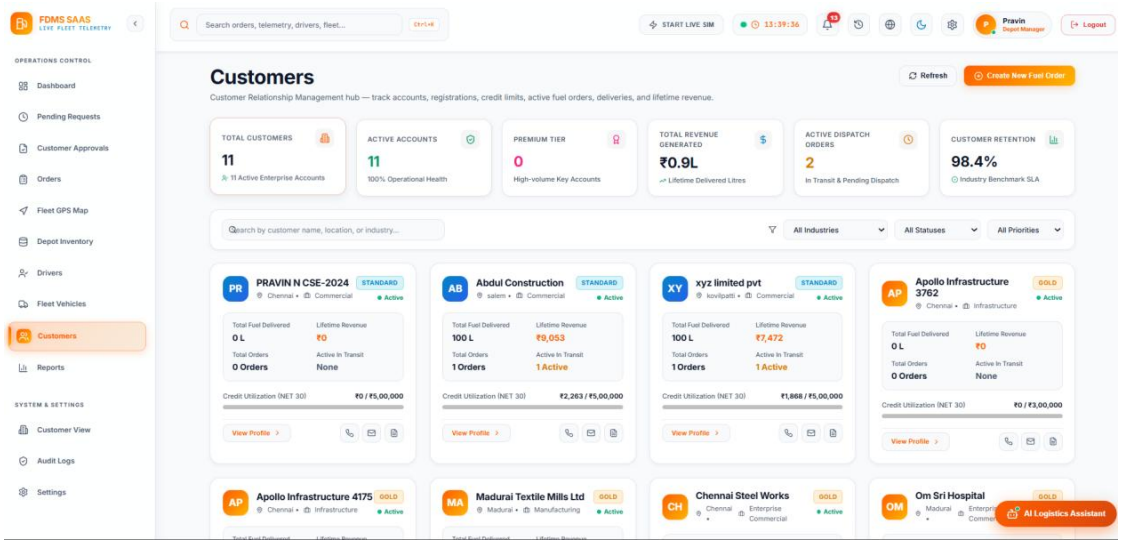


Figure 8.15: Corporate Customers Directory and Account Verification Overview

8.7 NOTIFICATIONS, AUDIT LOGS & REPORTS

The Notifications, Audit Logs & Reports module delivers system observability, compliance tracking, and business intelligence. It broadcasts real-time operational alerts across WebSockets, preserves an immutable audit trail of critical events, and generates multi-dimensional executive reports.

Purpose and Scope

Operational supply chain logistics requires immediate event awareness and verifiable accountability. This module generates targeted notifications whenever key events occur (order placed, approved, dispatched, delivered, low stock alarm) and streams them to connected users via Socket.IO. Simultaneously, an immutable audit logging service records every state-altering action with the user ID, role, action, target entity, IP address, and timestamp. The analytics engine aggregates historical data into revenue charts, fuel breakdown summaries, and exportable reports.

Functional Requirements

Real-Time Event Notifications: Pushes instant alerts to user and role rooms via Socket.IO; tracks read/unread status in database.

Immutable System Audit Trail: Records every security and operational event (Login, Order Approved, Stock Refilled) with user metadata.

Audit Log Search & Filters: Enables Auditors and Admins to filter logs by user role, action type, date range, and keyword.

Executive Delivery Analytics: Aggregates revenue, fuel volume by category, order counts, and top customers with CSV/Excel export.

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Event Notifications	~500 alerts / day	Order status changes, approvals, dispatch alerts, and stock threshold warnings.
Audit Log Records	~1,200 records / day	Captures user logins, approvals, stock modifications, and profile changes.
Analytical Query Load	50 - 100 queries / day	Depot manager dashboards, executive analytics, and end-of-day reconciliation.

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
GET	/api/notifications	All Roles (Auth)	Retrieves notification feed for currently authenticated user/role.
PUT	/api/notifications/:id/read	All Roles (Auth)	Marks specified notification as read in MongoDB.
GET	/api/activity-logs	Admin, Auditor	Queries immutable audit trail with search, role, and action filters.
GET	/api/reports/analytics	Depot Manager, Admin	Generates aggregated revenue, fuel volume, and delivery metrics.

HTTP Method	REST Endpoint	Role Access	Description & Payload
GET	/api/reports/export	Depot Manager, Admin	Exports comprehensive operational and dispatch reports as CSV/Excel.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy
notifications	Lookups by userId, role, and read status	Index on userId; index on role; index on createdAt descending.
activitylogs	Filtered queries by action, userRole, and timestamp	Compound index on (action, createdAt descending); index on userId.

Operational Interfaces and Evidence

Figure 8.16 displays the Executive Delivery Analytics & Reports interface (`Reports.jsx`), featuring interactive KPI tabs, revenue metrics, fuel volume breakdown charts, and report export controls.

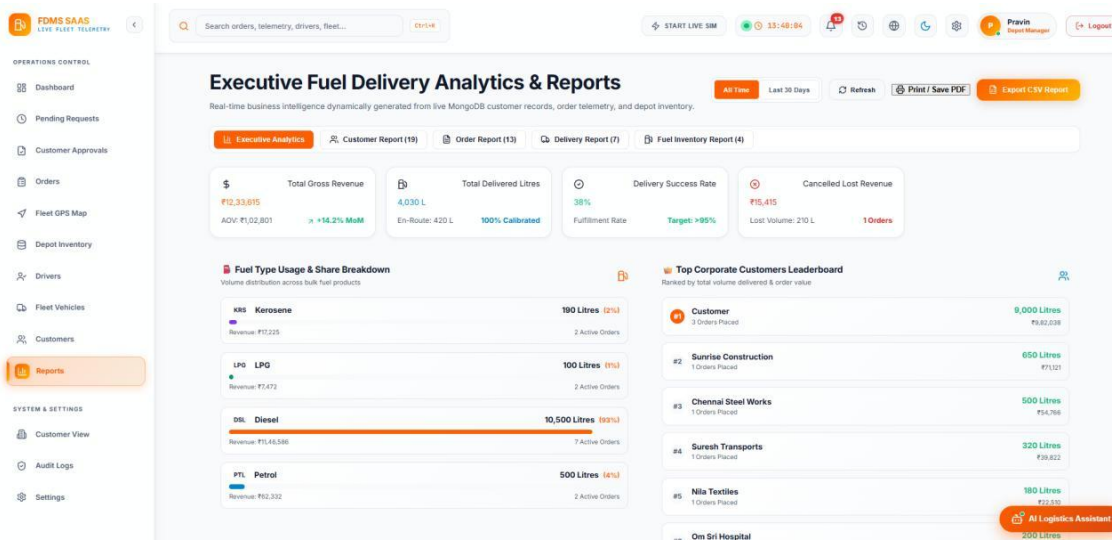


Figure 8.16: Executive Fuel Delivery Analytics and Comprehensive Reporting Interface

Figure 8.17 illustrates the System Audit Logs and Activity Trail interface (`AuditLogs.jsx`), presenting a searchable, verifiable chronological record of all user operations, action types, timestamps, and IP addresses.

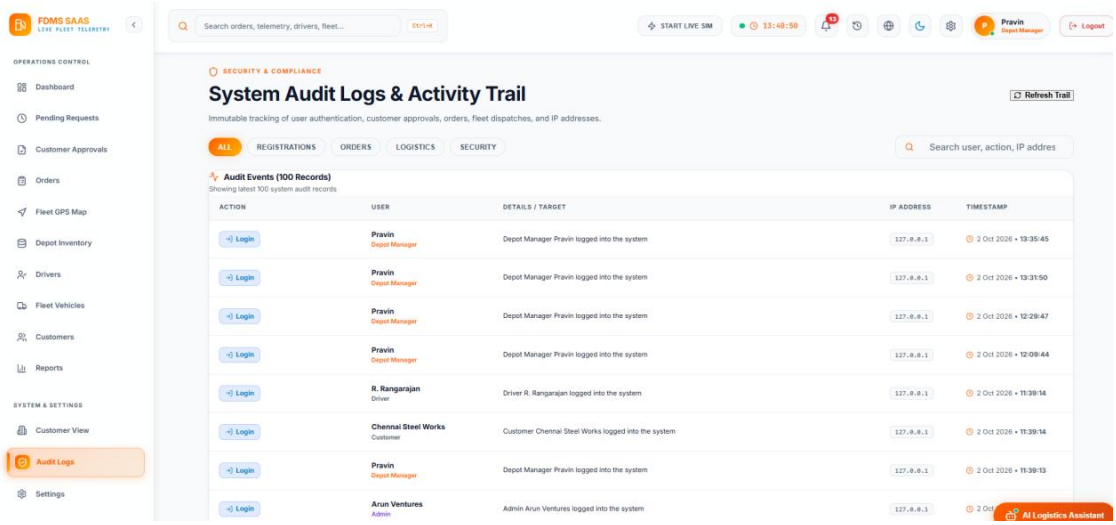


Figure 8.17: System Audit Logs and Compliance Activity Trail Interface

8.8 DASHBOARD, SETTINGS, LOCALISATION & ASSISTANT

The Dashboard, Settings, Localisation & Assistant module delivers customized user experiences across all six stakeholder roles. It provides role-specific dashboards, responsive visual themes, multilingual accessibility, and an operational AI logistics assistant.

Purpose and Scope

Different operational roles require distinct interface layouts. The Depot Manager requires operational throughput metrics (pending approvals, vault stock, in-transit tankers), while the Customer needs a streamlined ordering portal, and the Administrator requires enterprise governance controls. The module also manages user preferences (dark/light themes, alert sounds) and full interface localisation supporting English, Tamil, and Hindi. The integrated Fuel Logistics Assistant answers operational inquiries using live database state.

Functional Requirements

Role-Specific Dashboards: Renders tailored operational overviews: Depot Operations Center, Admin Dashboard, Customer Portal, Driver Portal.

Multilingual Localisation: Provides client-side translation dictionaries supporting English, Tamil, and Hindi, switchable instantly.

Theme Switching: Supports light and dark UI themes with preference persistence in browser `localStorage`.

Fuel Logistics Assistant: Interactive floating assistant analyzing live orders and tanks to provide rapid operational answers.

Workload Scaling Estimation

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Dashboard Page Views	1,500 - 3,000 views / day	Frequent refreshes and live socket updates across operational workstations.

Metric / Parameter	Estimated Daily Volume	Sizing & Capacity Basis
Localisation Dictionary Size	~45 KB total	Lightweight JSON translation dictionaries cached in browser client bundle.
Assistant Inquiries	100 - 250 queries / day	Fast in-memory rule evaluation against active order and inventory states.

API Interface Design

HTTP Method	REST Endpoint	Role Access	Description & Payload
GET	/api/dashboard/stats	All Roles (Auth)	Fetches aggregated KPI metrics tailored to role of requesting user.
GET	/api/settings	All Roles (Auth)	Retrieves user interface preferences, notification triggers, and active locale.
PUT	/api/settings	All Roles (Auth)	Updates user theme, language choice, and notification alert preferences.

Database Schema Design

Collection Name	Access Pattern	Indexing Strategy
users	Lookups by _id for settings persistence	Primary key index on _id.
orders, tanks, vehicles	Aggregated metric reads for dashboard cards	Indexed on status, fuelType, and createdAt.

Operational Interfaces and Evidence

Figure 8.18 showcases the Depot Operations Control Center (`Dashboard.jsx`), providing the Depot Manager with live operational cards, reserved fuel gauges, pending approvals, and active tanker fleet dispatches.

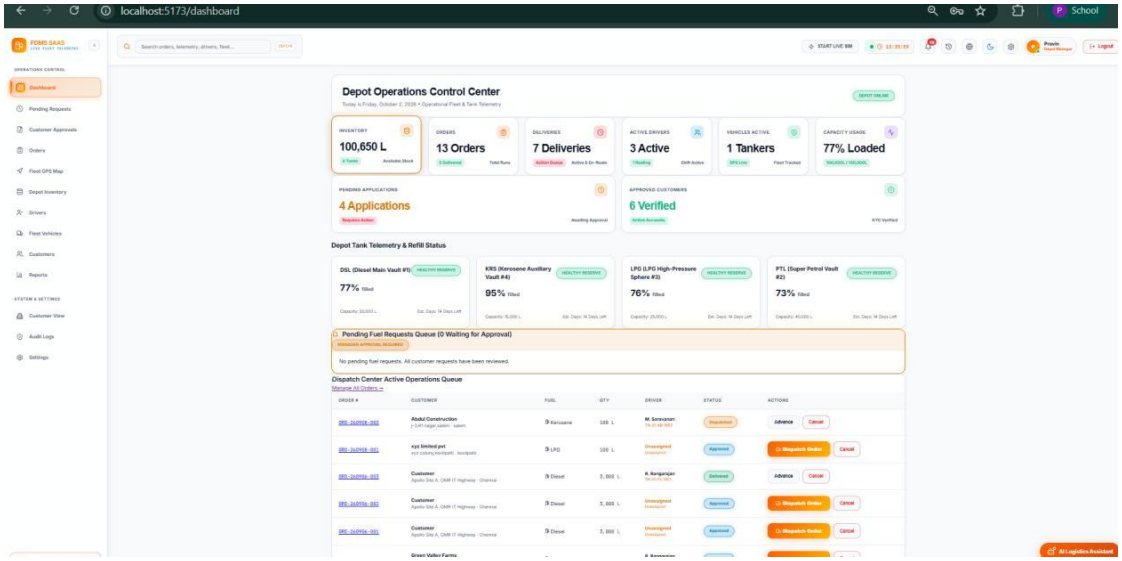


Figure 8.18: Depot Operations Control Center (Depot Manager Operational Dashboard)

Figure 8.19 displays the Administrator Governance Dashboard, highlighting system-wide user counts, total customer accounts, pending KYC verifications, and platform health metrics.

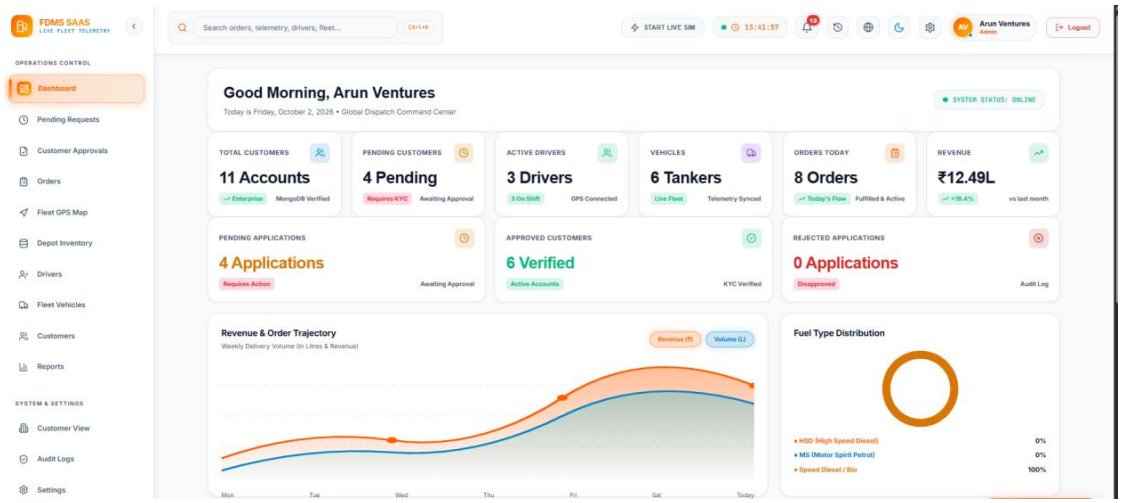


Figure 8.19: Administrator Enterprise Governance and Platform Overview Dashboard

Figure 8.20 displays the System Settings Portal (`Settings.jsx`), allowing users to toggle visual themes, select display language (English, Tamil, Hindi), configure audio alerts, and manage notification preferences.

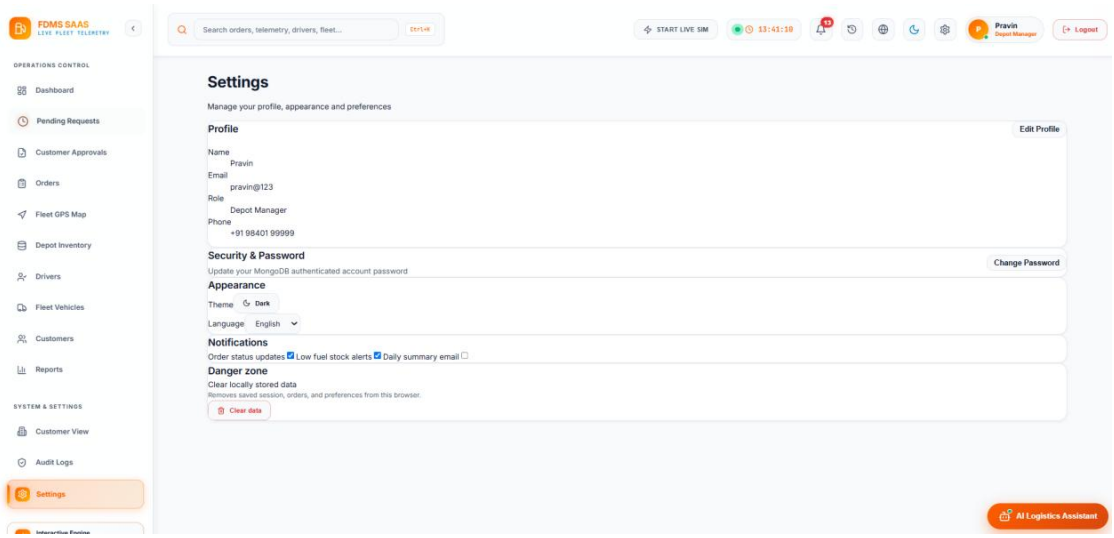


Figure 8.20: User Settings, Theme Configuration, and Multilingual Localisation Portal

8.9 DEPLOYMENT, CI/CD & TESTING

The Deployment, CI/CD & Testing module encompasses the build automation, continuous deployment pipelines, and automated software verification suites established for the Smart Fuel Dispatch & Fleet Telemetry System.

Purpose and Scope

To ensure production readiness, the repository incorporates automated build and deployment pipelines configured via Jenkins (`Jenkinsfile`) and Vercel (`vercel.json`). The testing infrastructure comprises nine specialized Node.js verification scripts that systematically exercise API endpoints, authentication flows, role-based access rules, approval lifecycles, and GPS telemetry streaming.

Jenkins CI/CD Pipeline Architecture

The Jenkins pipeline is defined in a declarative `Jenkinsfile` at the repository root, structuring continuous integration into four stages:

Checkout Stage: Pulls source code from the Git repository, checking out the latest commit on the production branch.

Install Stage: Installs dependencies across both root (`npm install`) and backend (`cd backend && npm install`) environments.

Build Stage: Executes the production Vite build (`npm run build`), generating bundled, tree-shaken static assets in `dist/`.

Deploy Stage: Deploys artifacts to the hosting infrastructure, updating environment configurations and restarting services.

Vercel Cloud Deployment Configuration

Production cloud hosting is configured via `vercel.json`, which defines a multi-service serverless architecture: static client assets are served from `dist/` via `@vercel/static`, while

all API requests matching `/api/(.*)` are dynamically routed to the Express server entrypoint (`backend/index.js`) using `@vercel/node`.

Automated Verification Test Suite

The project includes nine specialized Node.js automated test suites located in `backend/` that validate system correctness. Table 8.1 details the test suites and their verification results.

Test Suite Script	Focus & Target Module	Verification Scope & Outcome
test_e2e_flow.js	End-to-End Workflow	Simulates complete cycle: registration -> approval -> order -> stock reservation -> dispatch -> delivery (PASSED).
test_approval_workflow.js	Approval & KYC Logic	Validates customer application review, status transitions, and user provisioning (PASSED).
test_rbac_and_depots.js	RBAC Route Security	Exercises all endpoints with Admin, Manager, Customer, and Driver tokens; verifies 403 blocks (PASSED).
test_google_and_otp_auth.js	Multi-Modal Authentication	Tests Google ID token verification and mobile OTP generation/verification cycles (PASSED).
test_driver_location_flow.js	GPS Telemetry Ingestion	Transmits mock GPS coordinate streams; verifies Vehicle update and VehicleLocation log (PASSED).
test_all_roles_auth.js	Role Token Verification	Verifies JWT signing and claims validation across all six system roles (PASSED).
test_support_role_auth.js	Support Executive Role	Tests read-only and ticket handling permissions for Support Executive users (PASSED).
test_all_fixes.js	Regression Test Suite	Executes regression checks on alias fields, pricing calculations, and virtual getters (PASSED).
test_api.js	REST Health & Latency	Validates HTTP response codes, latency benchmarks, and CORS headers across routes (PASSED).

CHAPTER 9
CONCLUSION

The Smart Fuel Dispatch & Fleet Telemetry System brings corporate customer onboarding, fuel order processing, depot managerial approvals, automated inventory reservation, tanker dispatching, real-time GPS telemetry, underground storage tank monitoring, transparent pricing, instant notifications, immutable audit logging, and multi-role analytics into a unified, high-performance web platform. Built following the Model-View-Controller paradigm with a React 18 single-page frontend, a Node.js/Express 5 REST and WebSocket server, and a MongoDB Atlas cloud database, the platform demonstrates how modern web technologies can effectively eliminate operational friction, enhance logistical visibility, and prevent inventory loss in commercial petroleum supply chains.

Key Outcomes and Accomplishments

Multi-Modal Authentication & RBAC: Successfully implemented secure authentication supporting email/password with bcrypt hashing, Google OAuth 2.0 single sign-on, and mobile OTP login, enforced across six distinct user roles with route protection guards.

Verified Corporate Onboarding & KYC: Engineered a multi-step registration workflow with multipart document uploads via Multer, requiring explicit administrative approval before granting order placement privileges.

End-to-End Fuel Order Lifecycle: Implemented full transactional management—from automated quote generation and atomic stock reservation to tanker assignment, in-transit telemetry tracking, delivery proof submission, and automated digital invoicing.

Intelligent Proximity Dispatching: Integrated the Haversine great-circle formula to calculate Euclidean road proximity between delivery coordinates and active tankers, recommending optimal dispatch assignments.

Real-Time Telemetry & Geospatial Tracking: Established continuous GPS coordinate ingestion via REST and Socket.IO, streaming live vehicle positions to an interactive Leaflet map and preserving historical breadcrumb trails (up to 1,000 points per vehicle).

Depot Vault Inventory Telemetry: Automated underground tank monitoring with real-time tracking of capacity, volume, reserved allocations, and safety thresholds, preventing stock-outs through atomic reservations.

User Experience & Localisation: Delivered tailored dashboards for all roles, dark/light visual theme switching, internationalization supporting English, Tamil, and Hindi, and an operational AI logistics query assistant.

Production CI/CD & Automated Verification: Established automated build pipelines via Jenkins (`Jenkinsfile`), serverless multi-service deployment via Vercel (`vercel.json`), and nine specialized Node.js test scripts validating system correctness.

Current Operational Limitations

While the system fulfills all core academic and operational requirements, evaluation identifies specific areas for ongoing enhancement:

API Route Security Hardening: Currently, `protect` and `authorize` middleware are strictly applied to user profiles, passwords, user listings, and activity logs. In a commercial production rollout, all order, fleet, and inventory routes should require strict per-route token validation.

Administrative Role Segregation: Administrator and Depot Manager capabilities share operational overlap. Future iterations should enforce strict segregation between platform governance (user accounts, pricing, KYC approvals) and depot operations (orders, dispatch, tanks).

SMS Gateway Integration: OTP verification currently logs verification codes to the secure server console and database for testing; a production release requires integration with a licensed telecom SMS gateway (e.g., Twilio or MSG91).

Road-Network Routing & Traffic: Dispatch proximity calculations currently use straight-line Haversine Euclidean distance. Incorporating OpenStreetMap OSRM or Google Distance Matrix APIs would enable live traffic-aware routing and precise road distance calculations.

Future Scope and Scalability Enhancements

Cloud Object Storage Migration: Transitioning uploaded KYC documents from local disk storage to Amazon S3 or Google Cloud Storage with presigned URLs to facilitate fully stateless serverless hosting.

Time-Series Database Sharding: Migrating high-frequency GPS breadcrumb trails from standard collections to MongoDB Time-Series collections or InfluxDB with automated 30-day retention policies.

Integrated Payment Gateway: Embedding digital B2B commercial payment gateways (Razorpay, Stripe) for automated corporate escrow and net-banking settlement upon delivery confirmation.

Large Language Model Integration: Upgrading the rule-based Fuel Logistics Assistant with an LLM conversational agent connected via Retrieval-Augmented Generation (RAG) to live inventory and telemetry state.

In summary, the Smart Fuel Dispatch & Fleet Telemetry System successfully demonstrates the application of modern web engineering standards to bulk petroleum logistics. The resulting platform delivers an accessible, resilient, and verifiable solution that meets all requirements for the 23CS54C – Modern Web Technologies course.

REFERENCES

[1] Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd ed.). O'Reilly Media.

[2] Brown, E. (2019). Web Development with Node and Express: Leveraging the JavaScript Stack (2nd ed.). O'Reilly Media.

[3] Chodorow, K. (2019). MongoDB: The Definitive Guide - Powerful and Scalable Data Storage (3rd ed.). O'Reilly Media.

[4] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

[5] Resig, J., Bibeault, B., & Maras, J. (2016). Secrets of the JavaScript Ninja (2nd ed.). Manning Publications.

[6] Rai, R. (2013). Socket.IO Real-Time Web Application Development. Packt Publishing.

[7] Sinnatr, R. (2019). Leaflet.js Essentials: Create Interactive Web Maps with the Leaflet JavaScript Library. Packt Publishing.

[8] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF).

[9] Sinnott, R. W. (1984). Virtues of the Haversine. Sky and Telescope, 68(2), 159.

[10] Smart, J. F. (2011). Jenkins: The Definitive Guide - Continuous Integration for the Enterprise. O'Reilly Media.

[11] Mozilla Developer Network (MDN). (2024). Express Web Framework (Node.js/JavaScript). MDN Web Docs.

[12] React Documentation Team. (2024). React: The Library for Web and Native User Interfaces. Meta Open Source. https://react.dev/